

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
«Магнитогорский государственный технический университет  
им. Г.И. Носова»  
(ФГБОУ ВО «МГТУ им. Г.И. Носова»)

Кафедра вычислительной техники и программирования

## **ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА**

на тему:

**Проектирование и разработка информационной системы управления  
волонтерами с элементами геймификации и аналитики**  
по направлению подготовки 09.03.01 Информатика и вычислительная техника  
профиль «Проектирование и разработка Web-приложений»

Исполнитель: Зозин Павел Алексеевич, студент  
4 курса, группы Авб-22-2  
Руководитель: Гладышева Мария Михайловна

Магнитогорск, 2026

## ЗАДАНИЕ

на выполнение выпускной квалификационной работы

Обучающемуся Зозин Павел Алексеевич, группа Авб-22-2.

Тема работы: Проектирование и разработка информационной системы управления волонтерами с элементами геймификации и аналитики.

Исходные данные к работе: материалы технического задания, результаты преддипломной практики, программная реализация UnityAid, проектная документация, нормативные требования к оформлению ВКР.

Содержание расчетно-пояснительной записки:

- 1) проанализировать предметную область управления волонтерской деятельностью;
- 2) сформулировать требования к информационной системе и определить состав пользовательских ролей;
- 3) спроектировать архитектуру web-приложения, структуру базы данных и основные сценарии работы;
- 4) описать реализацию клиентской и серверной частей системы UnityAid;
- 5) провести проверку работоспособности и оценить результаты разработки.

Перечень графического материала: архитектура системы, схема пользовательского процесса, модель функциональных модулей, структура данных.

Дата выдачи задания: «22» апреля 2026 г.

Срок представления работы к защите: «\_\_\_» июня 2026 г.

Руководитель ВКР \_\_\_\_\_ /М.М. Гладышева/

Обучающийся \_\_\_\_\_ /П.А. Зозин/

## РЕФЕРАТ

Выпускная квалификационная работа содержит 3 раздела, введение, заключение, список использованных источников и приложения. В работе представлены таблицы с функциональными требованиями, структурой базы данных и результатами проверки, а также иллюстрации архитектуры и пользовательских процессов.

Ключевые слова: ИНФОРМАЦИОННАЯ СИСТЕМА, ВОЛОНТЕРЫ, WEB-ПРИЛОЖЕНИЕ, ГЕЙМИФИКАЦИЯ, АНАЛИТИКА, GO, VUE.JS, POSTGRESQL, REST API, СЕРТИФИКАТЫ

Объектом разработки является процесс управления волонтерской деятельностью в организациях, которые проводят мероприятия, распределяют задачи, учитывают вклад участников и формируют отчетность.

Цель работы - разработка web-ориентированной информационной системы UnityAid, позволяющей вести учет волонтеров, мероприятий, задач и подтвержденных часов, а также повышать вовлеченность участников за счет достижений, рейтингов и персональной статистики.

В ходе работы выполнен анализ предметной области, сформулированы функциональные и нефункциональные требования, спроектирована архитектура приложения, разработаны серверная часть на языке Go, клиентская часть на Vue.js и база данных PostgreSQL. Реализованы модули авторизации, организаций, мероприятий, задач, учета времени, уведомлений, базы знаний, геймификации, аналитики, сертификатов и системного администрирования.

Практическая значимость результата заключается в возможности использовать систему как основу для автоматизации деятельности малых и средних волонтерских организаций, студенческих объединений и социальных проектов.

## **СОДЕРЖАНИЕ**

Оглавление обновляется автоматически

## ВВЕДЕНИЕ

Развитие цифровых сервисов изменило подход к организации общественных, образовательных и социальных инициатив. Волонтерские объединения все чаще работают не как разовые группы помощи, а как устойчивые организационные структуры, которым требуется учет участников, планирование мероприятий, распределение задач, подтверждение часов и подготовка отчетов. При отсутствии единой информационной системы эти процессы распределяются между таблицами, мессенджерами, электронными формами и устными договоренностями.

Такая организация работы создает ряд практических проблем. Данные о волонтерах быстро устаревают, история участия хранится неполно, координатору сложно определить доступных исполнителей, а руководитель организации не получает оперативную картину по мероприятиям, задачам и фактически подтвержденному вкладу. Дополнительно возрастает нагрузка на сотрудников, которые вручную проверяют заявки, считают часы, готовят справки и сертификаты.

Актуальность темы выпускной квалификационной работы обусловлена необходимостью разработки специализированной web-системы, которая объединяет основные процессы волонтерской деятельности в одном интерфейсе и поддерживает как операционную работу координатора, так и мотивацию добровольцев. Для волонтера важны понятный личный кабинет, история участия, достижения и уведомления. Для координатора важны инструменты управления мероприятиями, задачами, заявками и отчетностью. Для администратора организации важны контроль доступа, аналитика и прозрачность действий пользователей.

Объектом разработки является процесс управления волонтерской деятельностью в организации. Предметом разработки является web-ориентированная информационная система UnityAid, обеспечивающая учет волонтеров, организаций, мероприятий, задач, времени, достижений, сертификатов и аналитических показателей.

Целью выпускной квалификационной работы является повышение эффективности управления волонтерской деятельностью за счет проектирования и разработки информационной системы UnityAid с элементами геймификации и аналитики.

Для достижения цели необходимо решить следующие задачи:

- 1) провести анализ предметной области и выявить проблемы существующей организации волонтерской деятельности;
- 2) определить пользовательские роли, функциональные требования и ограничения системы;
- 3) спроектировать архитектуру web-приложения, серверного API и базы данных;
- 4) реализовать основные модули системы с разграничением доступа по ролям;
- 5) проверить работоспособность приложения на ключевых пользовательских сценариях;
- 6) оценить практическую значимость и направления дальнейшего развития системы.

Методами выполнения работы являются системный анализ, моделирование предметной области, проектирование базы данных, проектирование REST API, разработка web-приложения, ручное и функциональное тестирование пользовательских сценариев.

Практическая значимость работы состоит в создании программного продукта, который может использоваться как основа для внедрения в малых и средних волонтерских организациях, студенческих объединениях, благотворительных проектах и учреждениях, которым необходимы учет участников, прозрачная отчетность и повышение вовлеченности добровольцев.

# **1 ТЕОРЕТИЧЕСКИЙ АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И СУЩЕСТВУЮЩИХ РЕШЕНИЙ УПРАВЛЕНИЯ ВОЛОНТЕРСКОЙ ДЕЯТЕЛЬНОСТЬЮ**

## **1.1 Описание предметной области**

Волонтерская деятельность за последние годы приобрела особое значение как инструмент решения социальных, образовательных, культурных и организационных задач. В работе некоммерческих организаций, образовательных учреждений, муниципальных проектов и инициативных объединений волонтеры участвуют в мероприятиях, выполняют задачи, помогают координаторам и формируют устойчивое сообщество вокруг социально значимых целей.

Рост числа добровольческих инициатив приводит к усложнению процессов управления. Организации должны учитывать сведения о волонтерах, их навыках и интересах, планировать мероприятия, распределять задачи, подтверждать участие, фиксировать затраченное время и формировать отчетность. При малом количестве участников эти процессы могут выполняться вручную, однако при увеличении масштаба ручная модель быстро становится источником ошибок и задержек.

Основными участниками предметной области являются волонтеры, координаторы, администраторы организаций и руководители программ. Волонтеры подают заявки на мероприятия, выполняют поручения, фиксируют вклад и ожидают обратной связи. Координаторы организуют события, распределяют задачи, подтверждают участие и часы. Администраторы управляют пользователями, ролями и настройками организации. Руководители программ используют аналитические сведения для оценки эффективности деятельности.

Для предметной области характерна необходимость централизованного учета. Данные о мероприятиях, заявках, задачах, часах и достижениях должны быть связаны между собой. Если сведения хранятся в разных таблицах, формах и переписках, организация теряет целостную картину: невозможно быстро

определить активных участников, подтвердить вклад конкретного волонтера или подготовить объективный отчет за период.

Отдельной проблемой является мотивация и удержание волонтеров. При отсутствии прозрачной системы признания участники не всегда видят результаты своей работы. Это снижает вовлеченность и затрудняет формирование постоянного сообщества. Элементы геймификации, такие как достижения, значки, уровни, рейтинги и персональная статистика, позволяют визуализировать вклад волонтера и поддерживать культуру признания заслуг.

Развитие web-технологий и распространение мобильных устройств создают предпосылки для разработки единой платформы управления волонтерской деятельностью. Прогрессивное web-приложение, адаптивный интерфейс и клиент-серверная архитектура позволяют обеспечить доступ к системе как с рабочего компьютера координатора, так и со смартфона волонтера. Это особенно важно для молодежной аудитории и распределенных команд.

Таблица 1 - Основные сущности предметной области

| <b>Сущность</b> | <b>Назначение</b>                                       | <b>Пример данных</b>                                   |
|-----------------|---------------------------------------------------------|--------------------------------------------------------|
| Волонтер        | Участник добровольческой деятельности                   | ФИО, контакты, навыки, интересы, история участия       |
| Организация     | Пространство управления волонтерской программой         | Название, участники, роли, настройки, логотип          |
| Мероприятие     | Событие, требующее участия волонтеров                   | Дата, место, формат, лимит, заявки, посещаемость       |
| Задача          | Поручение, назначаемое одному или нескольким участникам | Описание, срок, приоритет, статус, исполнители         |
| Запись времени  | Фиксация затраченных волонтерских часов                 | Дата, длительность, основание, статус подтверждения    |
| Достижение      | Элемент признания и геймификации                        | Название, условие выдачи, баллы, пользователь          |
| Отчет           | Агрегированная управленческая информация                | Активность, часы, мероприятия, динамика, эффективность |



## **1.2 Анализ аналогов**

На сегодняшний день существует множество инструментов, которые частично могут применяться для управления волонтерской деятельностью. Однако большинство из них либо предназначено для других типов процессов, либо требует значительной адаптации. Аналоги можно разделить на несколько групп: универсальные CMS, корпоративные ERP- и ECM-платформы, сервисы управления проектами, платформы регистрации на мероприятия и специализированные добровольческие сервисы.

Универсальные CMS, например Wordpress и Joomla, обладают гибкостью и большим количеством расширений. На их основе можно создать информационный сайт, форму регистрации или простую базу публикаций. Однако такие системы не содержат специализированной бизнес-логики для волонтерской деятельности. Для поддержки заявок, учета часов, ролей координаторов, геймификации и аналитики требуется разработка дополнительных модулей, что усложняет сопровождение.

Корпоративные ERP- и ECM-платформы, а также системы электронного документооборота имеют развитые средства управления документами, проектами и внутренними процессами. Их сильными сторонами являются масштабируемость, поддержка регламентов и интеграции. При этом для небольших НКО и образовательных организаций такие решения часто оказываются избыточными по функциональности, дорогими во внедрении и недостаточно ориентированными на повседневную работу волонтеров.

Сервисы управления проектами позволяют создавать задачи, назначать исполнителей и контролировать статусы. Они полезны для внутренней координации команды, но не решают специфические задачи: учет волонтерских часов, подтверждение участия, публичные мероприятия, достижения, сертификаты и аналитику добровольческой активности.

Специализированные добровольческие платформы ближе к предметной области, но часть из них ориентирована прежде всего на публичный поиск мероприятий и привлечение участников. Для конкретной организации важны

собственные пространства, роли, закрытые мероприятия, внутренняя база знаний, отчеты, гибкие правила геймификации и возможность дальнейшего развития под локальные процессы.

Таблица 2 - Сравнение групп аналогов

| <b>Группа решений</b>        | <b>Преимущества</b>                                         | <b>Ограничения для задачи UnityAid</b>                                               |
|------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Универсальные CMS            | Гибкость, расширения, быстрый запуск информационного сайта  | Нет встроенной логики заявок, часов, ролей, достижений и аналитики                   |
| ERP/ECM-платформы            | Мощные инструменты документооборота и управления процессами | Высокая стоимость, избыточность, сложность внедрения для небольших организаций       |
| Сервисы управления проектами | Удобные задачи, исполнители, статусы и сроки                | Не учитывают мероприятия, волонтерские часы, сертификаты и мотивацию участников      |
| Сервисы мероприятий          | Регистрация участников и публичные страницы событий         | Недостаточно функций для долгосрочной истории волонтера и отчетности                 |
| Добровольческие платформы    | Ориентация на добровольчество и поиск мероприятий           | Не всегда поддерживают автономное пространство организации, аналитику и геймификацию |

Анализ аналогов показывает, что для решения поставленной задачи требуется специализированная система. Она должна изначально учитывать логику волонтерской деятельности: участие в мероприятиях, выполнение задач, подтверждение часов, выдачу достижений, формирование отчетов и разграничение доступа по ролям.

Ключевым отличием UnityAid должна стать связность данных. В системе заявка на мероприятие, посещаемость, запись времени, достижение и сертификат не существуют как разрозненные элементы. Они образуют единую цепочку, позволяющую отследить путь волонтера от регистрации до подтвержденного вклада и управленческой аналитики.

### **1.3 Организация процесса управления волонтерской деятельностью**

Процесс управления волонтерской деятельностью включает несколько взаимосвязанных этапов. На первом этапе организация формирует

пространство работы: определяет координаторов, добавляет участников, настраивает роли и справочники. На втором этапе координатор планирует мероприятие или задачу, указывает сроки, описание, формат, ограничения и требования к участникам.

После публикации мероприятия волонтеры подают заявки. Система должна сохранять статус каждой заявки, комментарии и дату подачи. Координатор рассматривает заявки, подтверждает участие, отклоняет неподходящие заявки или переводит часть участников в лист ожидания. Наличие статусов делает процесс прозрачным и позволяет участнику понимать свое текущее положение.

Во время проведения мероприятия или выполнения задачи фиксируется фактическое участие. После завершения координатор отмечает посещаемость и подтверждает количество часов. Если волонтер самостоятельно создает запись времени, она должна пройти проверку. Только подтвержденные часы могут учитываться в профиле, аналитике, рейтингах и сертификатах.

Следующий этап связан с мотивацией. На основании подтвержденных действий система может начислять достижения, баллы или уровни. Геймификация при этом должна быть встроена в реальные процессы, а не существовать отдельно от них. Достижение должно отражать конкретный вклад: участие, количество часов, выполнение задач, регулярность или инициативность.

Завершающим этапом является аналитика и отчетность. Руководитель или администратор получает агрегированные показатели: количество активных волонтеров, проведенных мероприятий, выполненных задач, подтвержденных часов, выданных сертификатов и динамику активности по периодам. Эти сведения используются для планирования будущих мероприятий и оценки эффективности программ.

### Сценарий проведения мероприятия

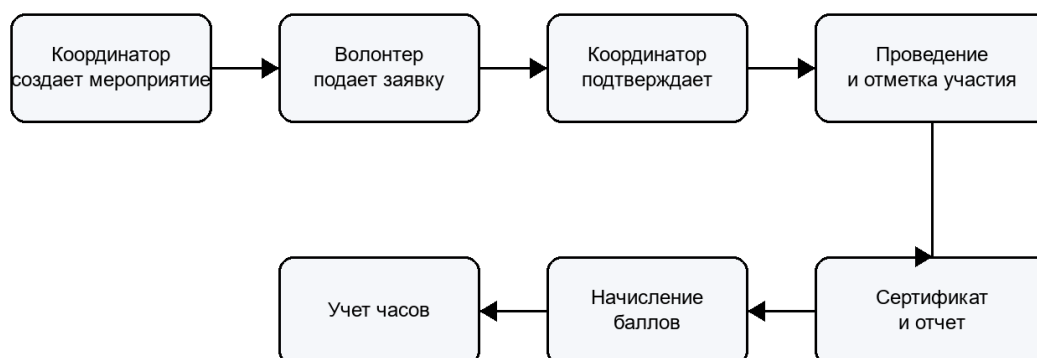


Рисунок 1 - Организация процесса управления волонтерской деятельностью

Таблица 3 - Роли пользователей в процессе управления

| Роль                      | Функции в процессе                                                       | Результат работы                                                   |
|---------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------|
| Волонтер                  | Подает заявки, выполняет задачи, фиксирует часы, получает достижения     | История участия, подтвержденные часы, сертификаты                  |
| Координатор               | Создает мероприятия и задачи, рассматривает заявки, подтверждает участие | Актуальные списки участников, корректные статусы, проверенные часы |
| Администратор организации | Управляет участниками, ролями, настройками и отчетами                    | Настроенное пространство организации и контроль данных             |
| Руководитель программы    | Анализирует показатели и принимает управленческие решения                | Сводная аналитика по активности и эффективности                    |

Технический уровень процесса реализуется через централизованную базу данных, серверное API и web-интерфейс. Все операции выполняются через систему, поэтому данные остаются актуальными и связанными. Такой подход снижает количество ошибок, характерных для ручного переноса сведений между таблицами, чатами и документами.

Для UnityAid важно поддерживать многоорганизационную модель. Один пользователь может состоять в нескольких организациях и иметь разные роли. Например, в одной организации он может быть координатором, а в другой -

обычным волонтером. Поэтому права доступа должны проверяться с учетом контекста организации, а не только глобального признака пользователя.

#### **1.4 Необходимость разработки системы UnityAid**

Необходимость разработки UnityAid обусловлена недостатками существующих решений и практическими потребностями небольших и средних волонтерских организаций. В реальной работе часто используются связки из таблиц, мессенджеров, онлайн-форм и ручных отчетов. Такая модель проста на старте, но плохо масштабируется и не обеспечивает достаточной надежности данных.

Типовыми проблемами являются отсутствие централизованного учета волонтеров и их активности, сложность планирования мероприятий, невозможность быстро отследить историю участия, низкая прозрачность признания заслуг и нехватка аналитических инструментов. Координаторы тратят значительное время на ручную сверку списков, рассылку уведомлений и подготовку отчетности.

UnityAid должна закрыть эти дефициты за счет единого web-приложения. В систему закладываются профили волонтеров, организации, мероприятия, задачи, учет времени, уведомления, база знаний, достижения, рейтинги, сертификаты и аналитика. Такой состав модулей отражает полный жизненный цикл волонтерской деятельности.

Централизация данных обеспечивает хранение сведений в единой базе с учетом ролей и прав доступа. Специализированная бизнес-логика позволяет поддерживать сценарии, характерные именно для волонтерства: подачу заявки, рассмотрение координатором, подтверждение участия, начисление достижений, фиксацию времени и формирование отчетов.

Многоорганизационный режим позволяет использовать одну систему для нескольких независимых организаций. Каждая организация получает собственных участников, мероприятия, задачи и настройки, а пользователь может работать в разных пространствах с разными ролями. Это важно для

образовательных учреждений, сетей НКО и проектов с несколькими направлениями.

Встроенная геймификация повышает вовлеченность участников. Значки, уровни и рейтинги не должны быть внешним декоративным элементом. Они должны опираться на подтвержденные действия и помогать волонтеру видеть накопленный вклад. Для координатора такая модель также полезна, поскольку позволяет выделять активных участников и поощрять регулярность.

Развитая аналитика необходима для управленческих решений. Сводные панели позволяют отслеживать количество активных волонтеров, закрытых задач, проведенных мероприятий, отработанных часов и выданных сертификатов. Эти показатели помогают оценивать эффективность программ, планировать нагрузку и готовить отчеты для руководства и партнеров.

Таким образом, разработка UnityAid обусловлена не только общей цифровизацией, но и конкретными дефицитами существующей практики управления волонтерской деятельностью. Специализированная web-ориентированная система позволяет объединить учет, координацию, мотивацию и аналитику в одном программном продукте.

### **1.5 Выводы по первой главе**

В первой главе рассмотрены особенности современной волонтерской деятельности, основные участники процесса и проблемы, возникающие при ручной организации работы. Показано, что разрозненные таблицы, мессенджеры и отдельные формы не обеспечивают достаточной прозрачности, усложняют подтверждение вклада и увеличивают нагрузку на координаторов.

Анализ предметной области позволил выделить базовые сущности, необходимые для эффективного управления: волонтеры, организации, мероприятия, задачи, записи времени, достижения и отчеты. Именно эти сущности должны стать ядром разрабатываемой информационной системы, поскольку они отражают полный жизненный цикл участия волонтера.

Рассмотрение аналогов показало, что универсальные CMS, ERP- и ECM-платформы, сервисы управления проектами и платформы мероприятий

закрывают только часть потребностей. Они либо требуют значительной доработки, либо являются избыточными, либо не поддерживают геймификацию, многоорганизационный режим и аналитику в нужном объеме.

На основании выполненного анализа обоснована необходимость разработки специализированной системы UnityAid. Первая глава формирует теоретическую основу проекта и задает требования, которые далее конкретизируются в архитектуре, модели данных, серверной и клиентской реализации.

## 2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ UNITYAID

### 2.1 Общая архитектура web-приложения

UnityAid спроектирована как web-приложение с разделением на клиентскую часть, серверное API и уровень хранения данных. Такой подход соответствует характеру задачи: пользователи работают через браузер, бизнес-логика сосредоточена на сервере, а данные хранятся в реляционной базе.

Клиентская часть реализована на Vue.js 3 с использованием TypeScript, Vite и Vue Router. Она отвечает за маршрутизацию, отображение страниц, формы ввода, таблицы, фильтры, навигацию и взаимодействие пользователя с API. Серверная часть реализована на Go и предоставляет REST API для модулей авторизации, организаций, мероприятий, задач, учета времени, уведомлений, аналитики и администрирования.

В качестве системы управления базами данных используется PostgreSQL. Выбор реляционной модели обусловлен большим количеством связей между пользователями, организациями, мероприятиями, задачами, заявками, часами и достижениями. Для локального запуска и проверки используется Docker Compose, что упрощает развертывание backend, frontend и базы данных в согласованной среде.

Архитектурное разделение также упрощает дальнейшее масштабирование. При росте нагрузки клиентская часть может обслуживаться отдельным web-сервером или CDN, серверное API может масштабироваться горизонтально, а база данных может быть вынесена на отдельный управляемый сервер. Для дипломного проекта достаточно локального контейнерного запуска, но структура не препятствует развитию.

REST API выбран как основной способ взаимодействия из-за простоты интеграции с web-интерфейсом и удобства отладки. Каждый предметный модуль имеет собственную группу маршрутов, а обмен данными выполняется в формате JSON. Такой подход понятен для frontend-разработки и хорошо соответствует CRUD-операциям над сущностями системы.



Отдельный worker-процесс предусмотрен для операций, которые не должны блокировать основной запрос пользователя. К ним относятся отправка уведомлений, подготовка отчетов, начисление достижений после завершения мероприятия и иные фоновые задачи. Даже если часть таких функций реализуется постепенно, архитектура заранее учитывает необходимость асинхронной обработки.

Архитектура UnityAid

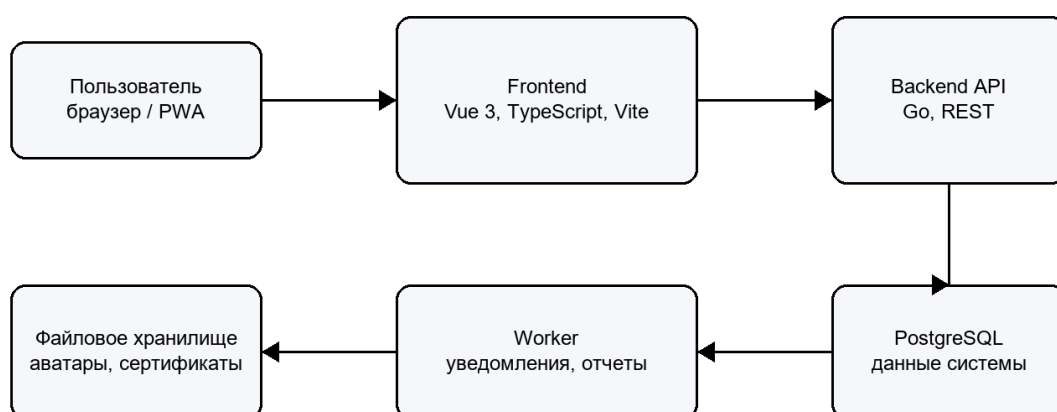


Рисунок 2 - Архитектура информационной системы UnityAid

## 2.2 Функциональные требования и пользовательские роли

Ролевая модель является основой разграничения доступа в системе. Один пользователь может состоять в разных организациях, а права определяются не только глобальной системной ролью, но и членством внутри конкретной организации. Это позволяет использовать систему как для одной организации, так и как основу для дальнейшего multi-tenant развития.

Неавторизованный пользователь имеет доступ только к входу, регистрации и восстановлению пароля. Волонтер получает доступ к личному кабинету, мероприятиям, задачам, часам, достижениям, уведомлениям и базе знаний. Координатор управляет операционными сущностями своей организации. Администратор организации управляет участниками и

настройками. Суперадминистратор работает с системной административной панелью.

Для каждой роли определяются не только разрешенные страницы, но и допустимые действия внутри страницы. Например, волонтер может видеть мероприятие и подать заявку, но не может подтвердить чужую заявку. Координатор может рассматривать заявки и отмечать посещаемость, но только в рамках организации, где он обладает соответствующей ролью.

Состояния сущностей также являются частью требований. Мероприятие может быть черновиком, опубликованным, завершенным или отмененным. Заявка может ожидать рассмотрения, быть подтвержденной, отклоненной, отмененной или находиться в листе ожидания. Задача имеет собственный жизненный цикл от создания до завершения. Явное хранение статусов делает процесс управляемым и проверяемым.

Для снижения ошибок пользовательский интерфейс должен подсказывать допустимые действия. Например, после завершения мероприятия координатору доступна отметка посещаемости и подтверждение часов, а действия по редактированию ключевых параметров должны быть ограничены. Это помогает сохранить целостность данных.

Таблица 3 - Функциональные требования по ролям

| <b>Роль</b>               | <b>Основные функции</b>                                        | <b>Ограничения</b>                                          |
|---------------------------|----------------------------------------------------------------|-------------------------------------------------------------|
| Гость                     | Регистрация, вход, восстановление пароля                       | Нет доступа к рабочим разделам                              |
| Волонтер                  | Профиль, заявки, задачи, часы, достижения, уведомления         | Не может управлять чужими данными и настройками организации |
| Координатор               | Мероприятия, заявки, задачи, посещаемость, подтверждение часов | Работает только в доступных организациях                    |
| Администратор организации | Участники, роли, данные организации, отчеты                    | Не управляет глобальными настройками приложения             |
| Суперадминистратор        | Системная админ-панель, организации, глобальные сущности       | Используется для технического администрирования             |

#### Функциональные модули системы

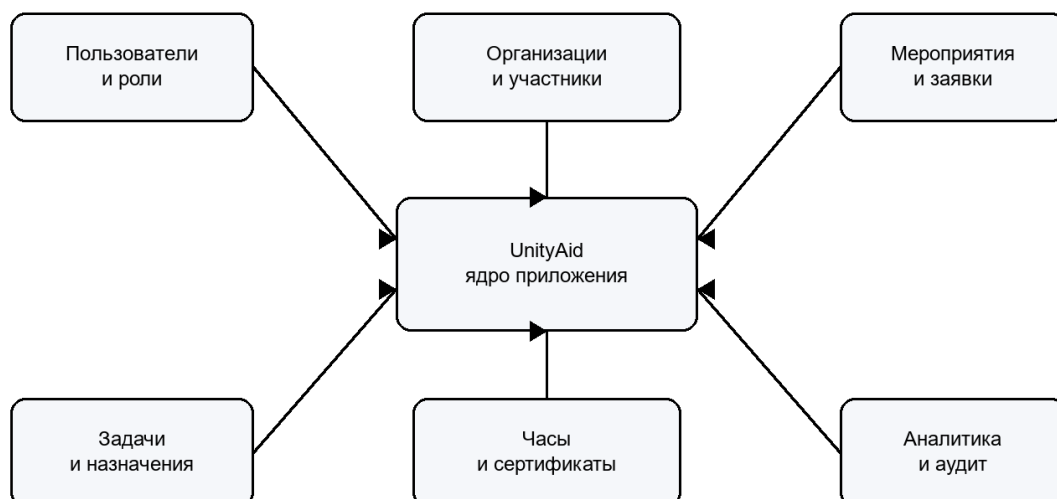


Рисунок 3 - Функциональные модули системы UnityAid

### 2.3 Проектирование базы данных

База данных проектировалась исходя из необходимости хранить как справочную, так и операционную информацию. Центральными сущностями являются `users`, `organizations`, `organization_members`, `volunteer_profiles`, `events`, `tasks`, `time_entries`, `achievements`, `notifications`, `certificates` и `audit_log`. Между ними существуют устойчивые связи, отражающие реальные процессы системы.

Таблица `users` хранит учетные записи и базовые данные пользователя. Профиль волонтера вынесен в отдельную сущность, что позволяет отделить данные авторизации от предметных характеристик: навыков, интересов, контактной информации и истории участия. Организации связаны с пользователями через таблицу членства, где фиксируется роль участника внутри организации.

Мероприятия и задачи привязаны к организациям. Заявки на мероприятия позволяют хранить статус участия, комментарии и решения координатора. Учет времени связывается с пользователем, организацией, мероприятием или задачей и проходит через подтверждение. Такой подход дает возможность формировать отчеты по разным срезам: по волонтеру, мероприятию, организации, периоду и статусу.

При проектировании учитывалась необходимость восстановления истории. Поэтому для ряда сущностей используются служебные поля даты создания, обновления и мягкого удаления. Мягкое удаление позволяет скрыть запись из интерфейса, но сохранить возможность аудита и анализа, если запись уже участвовала в связанных процессах.

Для справочников навыков, интересов, категорий новостей и категорий базы знаний используется отдельное хранение. Это позволяет администраторам постепенно настраивать систему под конкретную организацию, не меняя программный код. В перспективе такие справочники могут быть частично глобальными и частично локальными для организации.

Индексы и внешние ключи должны обеспечивать быстрый поиск по наиболее частым сценариям: список мероприятий организации, задачи пользователя, записи времени за период, заявки по мероприятию, уведомления текущего пользователя. Это особенно важно для страниц со списками и фильтрами.

Таблица 4 - Основные сущности базы данных

| Сущность      | Назначение                                        | Основные связи                                                  |
|---------------|---------------------------------------------------|-----------------------------------------------------------------|
| users         | Учетные записи и данные авторизации               | Связана с профилем, членством, задачами, часами, уведомлениями  |
| organizations | Волонтерские организации и настройки              | Связана с участниками, мероприятиями, задачами, сертификатами   |
| events        | Мероприятия, смены, заявки и посещаемость         | Связана с организацией, заявками, отзывами, часами              |
| tasks         | Задачи и назначения исполнителям                  | Связана с организацией, назначениями, комментариями, часами     |
| time_entries  | Записи волонтерских часов и статусы подтверждения | Связана с пользователем, организацией, мероприятием или задачей |
| achievements  | Правила достижений и выданные достижения          | Связана с пользователями и транзакциями баллов                  |
| certificates  | Сертификаты и справки о волонтерской деятельности | Связана с пользователем и организацией                          |
| audit_log     | Журнал значимых действий пользователей            | Связан с пользователем и изменяемыми сущностями                 |

## 2.4 Проектирование серверной части

Серверная часть реализует бизнес-логику приложения и предоставляет REST API. Код организован по модулям: auth, users, organizations, events, tasks, timeentries, notifications, gamification, analytics, certificates, knowledge, admin и другим. Для каждого модуля выделяются доменные структуры, репозиторий, сервис и HTTP-обработчик.

Разделение на слои повышает сопровождаемость проекта. Обработчик отвечает за прием HTTP-запроса и формирование ответа. Сервис содержит бизнес-правила: проверку прав, изменение статусов, начисление баллов, подтверждение часов. Репозиторий инкапсулирует SQL-запросы и работу с базой данных. Такой подход упрощает тестирование и снижает связность между транспортным уровнем и хранилищем.

Авторизация основана на пользовательской сессии и проверке ролей. Для операций изменения данных сервер проверяет не только факт входа, но и принадлежность пользователя к организации, в рамках которой выполняется действие. Например, координатор может изменить заявку только для мероприятия своей организации, а администратор организации не получает автоматически системные права суперадминистратора.

Важным элементом серверной части является аудит. Для операций создания, изменения и удаления записываются события, позволяющие восстановить историю действий. Это особенно важно для систем, где подтверждаются часы и формируются документы, поскольку организация должна понимать, кто и когда изменил статус записи.

Обработка ошибок строится так, чтобы клиентская часть могла показать пользователю понятное сообщение. Ошибки валидации отделяются от ошибок доступа и внутренних ошибок сервера. Например, отсутствие обязательного поля должно возвращать один тип ответа, а попытка изменить данные чужой организации - другой.

Для операций со списками используется страничная выдача. Это предотвращает загрузку слишком большого объема данных в одном запросе и

делает интерфейс стабильнее при росте числа пользователей, мероприятий или задач. Параметры поиска и фильтрации передаются через query-параметры API.

Отдельное внимание уделяется сертификатам. Сервер формирует документ, сохраняет сведения о нем и присваивает проверочный код. Публичный endpoint проверки не раскрывает лишние персональные данные, но позволяет подтвердить подлинность документа и его связь с организацией.

Таблица 5 - Основные группы API

| Группа API     | Назначение                                                                 |
|----------------|----------------------------------------------------------------------------|
| /auth          | Регистрация, вход, обновление сессии, выход, восстановление и смена пароля |
| /organizations | Создание организаций, управление участниками и ролями                      |
| /events        | Мероприятия, заявки, посещаемость, смены и обратная связь                  |
| /tasks         | Задачи, назначения, комментарии, вложения и история статусов               |
| /time-entries  | Создание, редактирование, подтверждение и отклонение часов                 |
| /analytics     | Отчеты по активности, мероприятиям, задачам, геймификации и аудиту         |
| /certificates  | Генерация, скачивание и публичная проверка сертификатов                    |
| /admin         | Универсальная системная административная панель                            |

## 2.5 Проектирование клиентской части

Клиентская часть UnityAid построена как одностраничное приложение. Основная навигация размещена в боковом меню, а верхняя панель содержит элементы профиля, уведомлений и быстрого доступа. Такой интерфейс подходит для регулярной работы координаторов и администраторов, поскольку основные разделы доступны из единого рабочего пространства.

Страницы приложения соответствуют предметным модулям: DashboardPage, EventsPage, EventDetailPage, TasksPage, TaskDetailPage, VolunteersPage, OrganizationsPage, TimeEntriesPage, AnalyticsPage, AchievementsPage, CertificatesPage, KnowledgeBasePage, NotificationsPage и AdminPanelPage. Для обмена с сервером используются отдельные API-слои внутри entities, что уменьшает дублирование запросов в компонентах.

Формы ввода реализуются через переиспользуемые компоненты. Для списков используются постраничная навигация, фильтры и поиск. Для ролей с ограниченным доступом интерфейс скрывает недоступные действия, однако окончательная проверка прав остается на сервере. Это важно, поскольку клиентская проверка улучшает удобство, но не является механизмом безопасности.

Особое внимание уделено адаптивности. Система должна быть доступна не только с рабочего компьютера координатора, но и с ноутбука или мобильного устройства волонтера. Поэтому интерфейс не опирается на сложные таблицы там, где пользователю достаточно карточек, списков и компактных форм.

Для состояния авторизации используется централизованное хранилище. Это позволяет не передавать данные текущего пользователя вручную между страницами и единообразно реагировать на истечение сессии. При загрузке приложения выполняется проверка текущего пользователя, после чего маршрутизатор определяет доступность разделов.

API-клиенты в папке сущностей отвечают за конкретные запросы к backend. Такой подход делает компоненты интерфейса проще: страница вызывает функцию, передает параметры и получает типизированный результат. При изменении endpoint достаточно обновить соответствующий API-модуль.

Для страниц с большим количеством записей используются фильтры и пагинация. Например, список задач можно ограничить по статусу, организации или поисковой строке. Это важно для координаторов, которые ежедневно работают с оперативными списками и должны быстро находить нужную запись.

Визуальный стиль интерфейса выбран рабочим и сдержанным. Для системы управления важнее читаемость, предсказуемая навигация и плотность полезной информации, чем декоративность. Основной экран должен помогать пользователю принять решение и выполнить действие за минимальное число шагов.

## **2.6 Реализация модулей аналитики, геймификации и сертификатов**

Модуль аналитики предназначен для получения управленческой информации. Он агрегирует данные по волонтерам, мероприятиям, задачам, подтвержденным часам, начисленным достижениям и действиям пользователей. Наличие аналитики позволяет руководителю организации видеть не только отдельные записи, но и динамику активности.

Геймификация реализуется через достижения, баллы и лидерборд. Система может начислять достижения за участие в мероприятиях, выполнение задач, накопление часов и регулярную активность. При этом геймификация рассматривается как вспомогательный механизм обратной связи, а не как замена содержательной мотивации волонтера.

Модуль сертификатов формирует документы, подтверждающие участие и вклад пользователя. Сертификат связывается с организацией и пользователем, имеет код проверки и может быть скачан в PDF. Публичная проверка по коду позволяет внешнему получателю убедиться, что документ действительно был сформирован системой.

Связка аналитики, геймификации и сертификатов повышает ценность системы. Волонтер получает видимый результат участия, координатор получает инструмент подтверждения вклада, а организация получает основание для отчетности и принятия управленческих решений.

Аналитика строится на уже накопленных операционных данных, поэтому ее точность зависит от корректности процессов подтверждения. Неподтвержденные часы не должны попадать в итоговые показатели наравне с подтвержденными. Это правило делает отчеты надежнее и снижает риск завышения активности.

Достижения проектируются как расширяемый справочник правил. В базовой версии достаточно достижений за количество часов, участие в мероприятиях и выполнение задач. В дальнейшем правила могут учитывать регулярность участия, направления волонтерства, качество обратной связи или работу в команде.



Сертификаты и справки являются важной частью практической ценности системы. Для студента или волонтера документ подтверждает вклад, а для организации упрощает оформление результатов. Проверочный код снижает риск подделки документа и позволяет внешнему получателю выполнить самостоятельную проверку.

Таблица 6 - Реализованные модули UnityAid

| Модуль       | Результат реализации                                                                 |
|--------------|--------------------------------------------------------------------------------------|
| Авторизация  | Регистрация, вход, refresh-сессии, выход, восстановление и смена пароля              |
| Организации  | Управление организациями, участниками, ролями, логотипом и данными                   |
| Мероприятия  | Создание, редактирование, заявки, статусы, смены, завершение, посещаемость           |
| Задачи       | Создание задач, назначения, статусы, комментарии, вложения, история изменений        |
| Учет времени | Создание записей, подтверждение и отклонение часов, связь с задачами и мероприятиями |
| База знаний  | Статьи, категории, публикация, редактирование и поиск материалов                     |
| Уведомления  | In-апп уведомления о событиях и изменениях статусов                                  |
| Сертификаты  | Генерация, скачивание и публичная проверка документов                                |
| Админ-панель | Постраничное управление системными сущностями для суперадминистратора                |

### 2.6.1 Описание предметных модулей

Модуль авторизации обеспечивает начальную точку входа в систему. Он отвечает за регистрацию, вход, обновление сессии, выход, восстановление пароля, подтверждение email и смену пароля. Для остальных модулей он предоставляет сведения о текущем пользователе и его правах.

Модуль пользователей и волонтерских профилей хранит сведения, необходимые для работы с участником. В профиле фиксируются контактные данные, навыки, интересы, история участия, подтвержденные часы и достижения. Разделение учетной записи и профиля позволяет не смешивать технические данные входа с предметной информацией.

Модуль организаций реализует пространство, внутри которого работают участники. Организация объединяет волонтеров, координаторов и

администраторов, а также является владельцем мероприятий, задач, новостей, сертификатов и аналитических показателей. Такая модель позволяет развивать систему в сторону нескольких независимых клиентов.

Модуль мероприятий поддерживает создание событий, публикацию, подачу заявок, рассмотрение заявок, учет смен, отметку посещаемости и завершение. Он является одним из центральных модулей, потому что через мероприятия проходит значительная часть волонтерской активности.

Модуль задач предназначен для распределения поручений между участниками. Задача имеет приоритет, срок, статус, исполнителей, комментарии, вложения и историю изменений. Это позволяет использовать систему не только для мероприятий, но и для постоянной операционной работы организации.

Модуль учета времени связывает вклад волонтера с конкретным основанием: мероприятием, задачей или иной деятельностью внутри организации. Подтверждение часов координатором отделяет заявленные сведения от проверенных, что повышает достоверность отчетности.

Модуль уведомлений информирует пользователя о значимых изменениях: статусе заявки, назначении задачи, подтверждении часов, появлении новых материалов или административных действиях. Наличие уведомлений снижает зависимость процессов от внешних мессенджеров.

Модуль базы знаний позволяет организации хранить инструкции, правила участия, ответы на частые вопросы и обучающие материалы. Для новых волонтеров это снижает порог входа, а для координаторов уменьшает количество повторяющихся объяснений.

Модуль аналитики формирует управленческие отчеты на основе операционных данных. Он помогает оценить активность волонтеров, эффективность мероприятий, выполнение задач, начисление достижений и действия пользователей. Аналитика делает систему полезной для руководителя, а не только для координатора.

Модуль сертификатов завершает цикл подтверждения вклада. Документ формируется на основе проверенных данных, связывается с пользователем и организацией, получает код проверки и может быть передан внешнему получателю. Это повышает практическую ценность системы для образовательной и социальной среды.

### **2.6.2 Обеспечение безопасности и целостности данных**

Безопасность UnityAid рассматривается на нескольких уровнях: защита учетных записей, разграничение доступа, валидация входных данных, контроль операций изменения и защита персональных сведений. Для системы, работающей с данными волонтеров, эти вопросы имеют практическое значение, поскольку в профилях могут храниться контакты, история участия и сведения о подтвержденных часах.

Пароли пользователей не должны храниться в открытом виде. При регистрации и смене пароля сервер сохраняет только криптографический хэш. При входе пользователь передает пароль по защищенному соединению, сервер сравнивает его с хэшем и выдает сессию только при успешной проверке. Такой подход снижает последствия возможной компрометации базы данных.

Разграничение доступа строится на проверке роли и принадлежности пользователя к организации. Это особенно важно для многоорганизационной модели. Пользователь может быть координатором в одной организации и обычным волонтером в другой, поэтому нельзя опираться только на глобальный признак роли. Каждая операция должна проверять контекст организации.

Валидация данных выполняется как на клиенте, так и на сервере. Клиентская валидация помогает пользователю быстрее исправить ошибку в форме, но не считается достаточной для защиты. Сервер повторно проверяет обязательные поля, допустимые значения статусов, формат дат, ограничения по лимитам участников и права пользователя на изменение записи.

Для сохранения целостности данных важно ограничивать переходы между статусами. Например, отмененное мероприятие не должно принимать

новые заявки, завершённое мероприятие не должно произвольно менять дату проведения, а подтверждённые часы должны изменяться только пользователем с соответствующими правами. Явные правила переходов делают поведение системы предсказуемым.

Журнал аудита фиксирует значимые операции: создание, изменение и удаление записей, изменение статусов, подтверждение часов, управление ролями. Наличие аудита повышает доверие к системе и помогает разбирать спорные ситуации. Если данные были изменены ошибочно, организация может установить, какое действие привело к проблеме.

### **2.6.3 Организация пользовательского интерфейса**

Интерфейс UnityAid проектировался как рабочее пространство, а не как рекламная страница. Основная задача пользователя - быстро перейти к нужному разделу, увидеть актуальные данные и выполнить действие. Поэтому навигация строится вокруг постоянного бокового меню, а страницы используют повторяемые элементы: заголовок, фильтры, список, карточку деталей и форму.

Для волонтера важны простые сценарии: увидеть доступные мероприятия, подать заявку, посмотреть назначенные задачи, добавить часы и получить уведомление о решении координатора. Эти действия не должны требовать знания внутренней структуры организации. Поэтому интерфейс волонтера скрывает административные детали и показывает только необходимые статусы.

Для координатора интерфейс должен быть более плотным. Ему необходимо работать со списками заявок, участников, задач и часов. Поэтому используются таблицы, фильтры, постраничная навигация и быстрые действия. При этом формы должны оставаться понятными, чтобы координатор мог создавать мероприятие или задачу без обращения к технической документации.

Для администратора организации важны настройки и контроль. На страницах организации отображаются участники, роли, основные сведения, логотип и управленческие действия. Такая структура позволяет разделить

повседневную операционную работу координатора и более редкие административные задачи.

Суперадминистраторская панель реализует универсальное управление сущностями. Она необходима не для ежедневной работы волонтерской организации, а для технической поддержки приложения, исправления данных и контроля системных справочников. Доступ к ней должен быть максимально ограничен.

Адаптивность интерфейса учитывает разные устройства. Координатор чаще работает за компьютером, но волонтер может использовать телефон. Поэтому ключевые пользовательские действия должны корректно отображаться на узких экранах, а сложные административные таблицы должны сохранять читаемость на рабочем экране.

Таблица 7 - Карта основных экранов клиентской части

| Экран             | Назначение                                      | Основные действия                                                |
|-------------------|-------------------------------------------------|------------------------------------------------------------------|
| LoginPage         | Вход пользователя в систему                     | Ввод email и пароля, переход к восстановлению доступа            |
| OnboardingPage    | Первичное знакомство и регистрационный сценарий | Регистрация, выбор начального действия                           |
| DashboardPage     | Сводная рабочая панель                          | Просмотр ключевых показателей и быстрых переходов                |
| EventsPage        | Список мероприятий                              | Фильтрация, просмотр, создание мероприятия координатором         |
| EventDetailPage   | Карточка мероприятия                            | Подача заявки, рассмотрение заявок, отметка посещаемости         |
| TasksPage         | Список задач                                    | Фильтрация, создание, назначение и изменение статуса             |
| TaskDetailPage    | Карточка задачи                                 | Просмотр описания, комментарии, вложения, учет времени           |
| TimeEntriesPage   | Учет волонтерских часов                         | Создание, редактирование, подтверждение и отклонение записей     |
| VolunteersPage    | Список волонтеров организации                   | Поиск, фильтрация, просмотр профиля и активности                 |
| OrganizationsPage | Список организаций                              | Просмотр доступных организаций и создание новой при наличии прав |

|                            |                                   |                                                                     |
|----------------------------|-----------------------------------|---------------------------------------------------------------------|
| Organization<br>DetailPage | Карточка<br>организации           | Управление участниками,<br>ролями, логотипом и<br>настройками       |
| AnalyticsPage              | Аналитика<br>организации          | Просмотр отчетов по<br>активности, мероприятиям,<br>задачам и часам |
| AchievementsPage           | Достижения и<br>рейтинг           | Просмотр личных<br>достижений и лидерборда                          |
| CertificatesPage           | Сертификаты и<br>справки          | Генерация, скачивание и<br>просмотр документов                      |
| CertificateVerifyPage      | Публичная проверка<br>сертификата | Проверка документа по<br>коду без входа в систему                   |
| KnowledgeBasePage          | База знаний                       | Просмотр статей, поиск и<br>переход к материалам                    |
| KnowledgeFormPage          | Форма статьи базы<br>знаний       | Создание и<br>редактирование материала                              |
| NewsListPage               | Новости<br>организации            | Просмотр объявлений и<br>публикаций                                 |
| NotificationsPage          | Уведомления<br>пользователя       | Просмотр событий и<br>отметка уведомлений<br>прочитанными           |
| AdminPanelPage             | Системное<br>администрирование    | Постраничное управление<br>разрешенными сущностями                  |

#### 2.6.4 Развертывание и сопровождение

Для локального запуска проекта используется Docker Compose. Он объединяет backend, frontend, базу данных и сопутствующие сервисы в единый сценарий запуска. Это снижает количество ручных действий и помогает воспроизводить окружение на другой машине.

Миграции базы данных позволяют пошагово изменять структуру таблиц. Это важно для проекта, который развивается итерационно: при добавлении сертификатов, уведомлений или новых связей не требуется вручную изменять схему базы. Миграции фиксируют историю изменений и позволяют развернуть проект с нуля.

Для production-развертывания предусмотрены отдельные Dockerfile и конфигурация прокси. Такой подход отделяет режим разработки от режима эксплуатации. В разработке важны быстрые перезапуски и удобство отладки, а в эксплуатации - стабильная сборка, предсказуемые переменные окружения и корректная маршрутизация запросов.

Сопровождение системы предполагает резервное копирование базы данных, контроль логов, обновление зависимостей и проверку доступности сервиса. Даже если в рамках дипломной работы эти процессы описаны на уровне проектного решения, их учет показывает готовность архитектуры к реальному использованию.

Важным направлением сопровождения является управление клиентскими экземплярами. UnityAid может развиваться как коробочная система для отдельной организации или как SaaS-платформа. В обоих случаях необходимы процедуры создания организации, настройки домена, загрузки логотипа, выдачи начальных ролей и проверки работоспособности.

Таблица 8 - Эксплуатационные требования к системе

| <b>Требование</b>     | <b>Описание</b>                                                                  |
|-----------------------|----------------------------------------------------------------------------------|
| Резервное копирование | Регулярное сохранение базы данных и файловых вложений                            |
| Логирование           | Фиксация ошибок приложения и значимых действий пользователей                     |
| Мониторинг            | Контроль доступности API, базы данных и фоновых задач                            |
| Обновление            | Применение миграций и поставка новых версий без потери данных                    |
| Настройка клиента     | Создание организации, назначение ролей, загрузка логотипа и базовых справочников |

## 2.7 Выводы по второму разделу

Во втором разделе описаны архитектура, функциональные требования, модель данных и реализация ключевых частей UnityAid. Система построена как web-приложение с клиентской частью на Vue.js, серверной частью на Go и базой данных PostgreSQL.

Разработанная структура модулей обеспечивает поддержку основных процессов волонтерской организации: работу пользователей и ролей, мероприятия, задачи, учет времени, уведомления, базу знаний, достижения, аналитику, сертификаты и системное администрирование.

## 3 ПРОВЕРКА РАБОТОСПОСОБНОСТИ И ОЦЕНКА РЕЗУЛЬТАТОВ

### 3.1 Организация испытаний

Проверка работоспособности UnityAid выполнялась по основным пользовательским сценариям. Цель испытаний состояла в подтверждении того, что реализованные модули корректно взаимодействуют друг с другом и обеспечивают полный цикл работы: от регистрации пользователя до формирования отчетных данных и сертификата.

Backend проверяется запуском тестов, применением миграций на чистой базе и ручной проверкой API. Frontend проверяется сборкой проекта, переходами по основным страницам, заполнением форм и выполнением операций создания, редактирования и удаления сущностей. Отдельно проверяются права доступа, поскольку ошибки разграничения ролей могут привести к раскрытию или изменению чужих данных.

Испытания целесообразно проводить на подготовленном демонстрационном наборе данных. В него входят несколько пользователей с разными ролями, одна или несколько организаций, мероприятия в разных статусах, задачи, записи времени, достижения и сертификаты. Такой набор позволяет быстро проверить не только пустые формы, но и реальные списки.

Проверка backend начинается с миграций, потому что ошибки схемы базы данных проявляются во всех остальных модулях. После успешного применения миграций проверяются базовые endpoint авторизации, затем CRUD-операции и сценарии, где несколько сущностей взаимодействуют между собой.

Проверка frontend включает не только сборку, но и ручной проход по интерфейсу. Важно убедиться, что пользователь видит только допустимые пункты меню, формы не отправляют некорректные данные, ошибки API отображаются понятным образом, а после успешного действия список или карточка обновляются.

Таблица 9 - Основные проверки системы

| Направление | Проверяемые действия | Ожидаемый результат   |
|-------------|----------------------|-----------------------|
| Авториза    | Регистрация, вход,   | Пользователь получает |



|                  |                                                                      |                                                                  |
|------------------|----------------------------------------------------------------------|------------------------------------------------------------------|
| ция              | refresh, logout, смена пароля                                        | доступ только после успешной проверки учетных данных             |
| Организа<br>ции  | Создание организации,<br>добавление участника,<br>назначение роли    | Данные доступны в<br>пределах организации и с<br>учетом роли     |
| Мероприя<br>тия  | Создание мероприятия,<br>подача и рассмотрение заявки,<br>завершение | Статусы заявок и<br>мероприятия изменяются<br>корректно          |
| Часы             | Создание записи,<br>подтверждение, отклонение                        | Подтвержденные часы<br>учитываются в профиле и<br>отчетах        |
| Геймифик<br>ация | Начисление достижений<br>и баллов после активности                   | Пользователь видит<br>достижения и место в рейтинге              |
| Сертифик<br>аты  | Генерация, скачивание,<br>публичная проверка                         | Документ имеет<br>проверочный код и доступен<br>для проверки     |
| Аналитик<br>а    | Открытие отчетов и<br>дашборда                                       | Отображаются<br>агрегированные показатели по<br>доступным данным |

### 3.2 Демонстрационный сценарий работы системы

Демонстрационный сценарий отражает типовую работу волонтерской организации. Сначала суперадминистратор открывает систему и создает организацию либо использует уже существующую. Затем администратор организации добавляет пользователей и назначает координатора. Координатор создает мероприятие, указывает формат, дату, место, лимит участников и публикует его для волонтеров.

Волонтер входит в систему, открывает список мероприятий и подает заявку. Координатор рассматривает заявку и подтверждает участие. После проведения мероприятия координатор отмечает посещаемость и подтверждает часы. Система отражает часы в профиле пользователя, начисляет баллы и достижения, а затем позволяет сформировать сертификат.

В завершающей части сценария администратор открывает аналитический раздел и проверяет показатели активности. Суперадминистратор может открыть журнал аудита и убедиться, что операции создания, изменения и удаления были зафиксированы. Такой сценарий подтверждает связность модулей и показывает практическую применимость системы.

Сценарий также показывает, что система поддерживает разные уровни ответственности. Волонтер инициирует участие, но не может самостоятельно подтвердить часы. Координатор управляет мероприятием, но действует в пределах своей организации. Администратор видит более широкий контур и может управлять участниками. Суперадминистратор обслуживает систему в целом.

При демонстрации важно проверять не только положительный путь, но и ограничения. Например, волонтер не должен видеть административную панель, координатор не должен изменять данные другой организации, а неподтвержденная запись времени не должна увеличивать итоговые часы в сертификате.

Такая проверка позволяет выявить ошибки, которые не всегда обнаруживаются при изолированном тестировании отдельных endpoint. В реальной системе ценность создается именно связкой модулей, поэтому полный пользовательский сценарий является обязательной частью приемки.

- 1) суперадминистратор входит в систему и проверяет список организаций;
- 2) администратор организации добавляет участника и назначает координатора;
- 3) координатор создает мероприятие и публикует его;
- 4) волонтер подает заявку на участие;
- 5) координатор подтверждает заявку и отмечает посещаемость;
- 6) волонтер или координатор добавляет запись времени;
- 7) координатор подтверждает часы;
- 8) система начисляет баллы и достижения;
- 9) координатор формирует сертификат;
- 10) проверяющий открывает публичную страницу проверки сертификата.

### **3.3 Анализ результатов разработки**

В результате выполнения работы получена информационная система, реализующая основные процессы управления волонтерской деятельностью.

Разработка охватывает как операционные сценарии координатора, так и пользовательские сценарии волонтера. Наличие административной панели и аудита делает систему пригодной для использования не только как демонстрационного учебного проекта, но и как основы для практического внедрения.

К сильным сторонам реализации относятся модульная структура backend, разделение API-слоя и интерфейса, использование PostgreSQL для связанных данных, наличие миграций, поддержка Docker Compose, а также широкий набор предметных модулей. Система не ограничивается регистрацией на мероприятия, а поддерживает задачи, часы, достижения, уведомления, базу знаний, сертификаты и аналитику.

Ограничения текущей версии связаны с тем, что проект выполнялся в рамках выпускной квалификационной работы. Часть перспективных функций может быть расширена: интеграции с календарями и мессенджерами, расширенная настройка правил геймификации, полнотекстовый поиск, импорт больших списков волонтеров, развитая система шаблонов документов и мониторинг production-окружения.

С точки зрения архитектуры результат можно считать расширяемым. Новые модули могут добавляться через отдельные группы маршрутов backend, новые таблицы миграций и новые страницы frontend. При этом уже существующая модель организаций и ролей позволяет сохранять общий принцип разграничения доступа.

С точки зрения предметной области результат закрывает базовый управленческий цикл. Организация получает учет участников, мероприятий, задач и часов; волонтер получает личную историю и достижения; координатор получает инструменты ежедневной работы; руководитель получает аналитику и документы.

С точки зрения учебной ценности проект демонстрирует применение нескольких компетенций: анализ предметной области, проектирование базы

данных, разработка backend и frontend, работа с контейнерным окружением, проектирование интерфейса и подготовка документации.

Таблица 10 - Соответствие задач работы полученным результатам

| Задача                     | Полученный результат                                                                               |
|----------------------------|----------------------------------------------------------------------------------------------------|
| Анализ предметной области  | Выявлены роли, процессы, проблемы ручного учета и требования к системе                             |
| Проектирование архитектуры | Определена трехзвенная web-архитектура с REST API и PostgreSQL                                     |
| Проектирование базы данных | Выделены сущности пользователей, организаций, мероприятий, задач, часов, достижений и сертификатов |
| Разработка backend         | Реализованы модульные обработчики, сервисы и репозитории на Go                                     |
| Разработка frontend        | Созданы страницы и компоненты на Vue.js для основных пользовательских сценариев                    |
| Проверка работоспособности | Сформирована программа испытаний и проверены ключевые сценарии                                     |

### 3.4 Экономическая и практическая значимость

Практическая значимость UnityAid заключается в снижении трудозатрат координаторов на учет заявок, часов и отчетов. Если организация ведет процессы вручную, сотрудник регулярно тратит время на перенос данных между таблицами, сверку списков, подготовку справок и поиск истории участия. Единая система уменьшает число ручных операций и снижает риск ошибок.

Экономический эффект для небольшой организации выражается прежде всего в экономии рабочего времени. Например, автоматическое формирование списков участников, подтверждение часов и подготовка сертификатов позволяют координатору уделять больше внимания содержательной работе с добровольцами. Для образовательных организаций это также повышает прозрачность учета внеучебной активности студентов.

С точки зрения дальнейшего развития UnityAid может быть оформлена как коробочное решение для одной организации или как SaaS-платформа с изолированными пространствами организаций. Уже реализованные сущности организаций, членства и ролей создают основу для масштабирования продукта.

Дополнительную ценность могут дать настройки бренда, публичные страницы организаций, конструктор процессов согласования и расширенные отчеты.

Для оценки экономической целесообразности можно рассматривать сокращение времени на повторяющиеся операции. Если координатор еженедельно вручную сводит заявки, посещаемость и часы, автоматизация даже части этих действий дает заметную экономию. При увеличении числа мероприятий эффект растет, потому что система повторно использует уже введенные данные.

Косвенный эффект выражается в повышении качества данных. Когда сведения вводятся один раз и используются в связанных модулях, уменьшается число расхождений между списками. Это особенно важно для отчетности, где ошибка в часах или участниках может привести к необходимости повторной проверки документов.

Практическая ценность также связана с удержанием волонтеров. Прозрачная история участия, достижения, уведомления и сертификаты помогают участнику видеть результат своей работы. Для организации это означает более устойчивое сообщество и меньшие затраты на постоянное привлечение новых людей.

Таблица 11 - Оценка сокращения ручных операций координатора

| <b>Операция</b>        | <b>Ручной подход</b>                                  | <b>Подход с UnityAid</b>                                |
|------------------------|-------------------------------------------------------|---------------------------------------------------------|
| Сбор заявок            | Форма, таблица и ручное уведомление участников        | Заявка создается в системе и получает статус            |
| Подтверждение участия  | Сверка списка и переписка с каждым участником         | Координатор меняет статус заявки в карточке мероприятия |
| Учет часов             | Ручной подсчет по посещаемости и отдельным сообщениям | Записи времени связаны с мероприятием или задачей       |
| Подготовка сертификата | Ручное заполнение шаблона и проверка данных           | Документ формируется по подтвержденным данным           |
| Отчетность             | Сведение нескольких таблиц за период                  | Показатели строятся на операционных данных системы      |

Даже без точного коммерческого расчета видно, что основной экономический эффект связан с повторяемостью операций. Чем больше мероприятий проводит организация, тем чаще координатор выполняет однотипные действия. Автоматизация таких действий дает накопительный эффект и уменьшает зависимость качества учета от внимательности одного сотрудника.

Дополнительная экономическая ценность связана с подготовкой документов. Если справки и сертификаты формируются вручную, сотрудник должен проверить ФИО, организацию, даты, количество часов и основание выдачи. В UnityAid эти данные уже связаны с пользователем, мероприятием и подтвержденными часами, поэтому риск ошибки ниже.

Для студенческого волонтерства важен не только прямой экономический эффект, но и управленческая прозрачность. Университет или факультет может видеть активность по группам, направлениям и периодам, а студент получает подтверждение своего участия. Это повышает качество внеучебной работы и облегчает подготовку отчетности.

Для благотворительных организаций система может быть полезна при подготовке отчетов перед партнерами. Наличие истории мероприятий, подтвержденных часов и сертификатов позволяет быстрее отвечать на вопросы о результатах программы и демонстрировать вклад добровольцев.

Развитие UnityAid как продукта может идти поэтапно. На первом этапе система используется как локальный инструмент одной организации. На втором этапе добавляются настройки бренда, шаблоны документов и расширенная аналитика. На третьем этапе возможен переход к SaaS-модели, где каждая организация работает в собственном пространстве.

В SaaS-модели важны биллинг, изоляция данных, управление тарифами, резервное копирование и мониторинг. Эти функции не являются обязательными для дипломного MVP, но текущая модель организаций, ролей и прав доступа создает основу для их последующего добавления.

### **3.5 Выводы по третьему разделу**

В третьем разделе описана организация проверки работоспособности UnityAid, представлен демонстрационный сценарий и выполнена оценка результатов разработки. Проверка показывает, что система поддерживает полный рабочий цикл: создание организации, управление участниками, проведение мероприятия, подтверждение часов, начисление достижений, генерацию сертификата и просмотр аналитики.

Полученный программный продукт имеет практическую значимость для автоматизации волонтерской деятельности и может развиваться в направлении коммерческого коробочного решения или multi-tenant SaaS-платформы.

## ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была спроектирована и разработана информационная система UnityAid, предназначенная для управления волонтерской деятельностью с элементами геймификации и аналитики. Работа включала анализ предметной области, постановку требований, проектирование архитектуры, разработку серверной и клиентской частей, проектирование базы данных и проверку основных сценариев.

В первом разделе рассмотрены особенности волонтерской деятельности как объекта автоматизации. Выявлены проблемы ручного учета: разрозненность данных, сложность подтверждения часов, отсутствие прозрачной истории участия, высокая нагрузка на координаторов и недостаток аналитики. На основании анализа обоснована необходимость создания специализированной системы.

Во втором разделе выполнено проектирование и описание реализации UnityAid. Система построена на клиент-серверной архитектуре: frontend реализован на Vue.js 3 и TypeScript, backend - на Go, база данных - PostgreSQL. В составе приложения реализованы модули авторизации, организаций, мероприятий, задач, учета времени, уведомлений, базы знаний, геймификации, аналитики, сертификатов и системного администрирования.

В третьем разделе рассмотрена проверка работоспособности и оценка результатов. Демонстрационный сценарий подтвердил возможность прохождения полного цикла работы: от создания организации и мероприятия до подтверждения часов, начисления достижений и формирования сертификата. Полученные результаты соответствуют поставленной цели и задачам.

Практическая значимость работы состоит в создании программной основы для автоматизации деятельности волонтерских организаций, студенческих объединений и социальных проектов. Дальнейшее развитие системы может быть связано с интеграциями календарей и мессенджеров,



расширением аналитики, настройкой правил геймификации, импортом данных и развитием multi-tenant модели.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления.
2. ГОСТ Р 7.0.5-2008. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая ссылка. Общие требования и правила составления.
3. ГОСТ Р 2.105-2019. Единая система конструкторской документации. Общие требования к текстовым документам.
4. СМК-О-СМГТУ-36-20. Выпускная квалификационная работа. Структура, содержание, общие правила выполнения и оформления.
5. Выпускная квалификационная работа: от бакалавриата до аспирантуры: учебное пособие / О.С. Логунова, Л.Г. Егорова, М.Ю. Наркевич, Ю.В. Кочержинская. Магнитогорск: МГТУ им. Г.И. Носова, 2025.
6. Федеральный закон от 11.08.1995 N 135-ФЗ «О благотворительной деятельности и добровольчестве (волонтерстве)».
7. Федеральный закон от 27.07.2006 N 152-ФЗ «О персональных данных».
8. Документация языка Go. URL: <https://go.dev/doc/>
9. Документация Vue.js. URL: <https://vuejs.org/guide/introduction.html>
10. Документация PostgreSQL. URL: <https://www.postgresql.org/docs/>
11. Документация Docker. URL: <https://docs.docker.com/>
12. Fielding R.T. Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation. University of California, Irvine, 2000.
13. Newman S. Building Microservices. Designing Fine-Grained Systems. O'Reilly Media, 2021.
14. Fowler M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
15. Martin R.C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.

16. OWASP Foundation. OWASP Top Ten Web Application Security Risks. URL: <https://owasp.org/www-project-top-ten/>
17. Документация Vite. URL: <https://vitejs.dev/guide/>
18. Документация TypeScript. URL: <https://www.typescriptlang.org/docs/>
19. Документация OpenAPI Specification. URL: <https://spec.openapis.org/oas/latest.html>
20. Документация JSON Web Tokens. URL: <https://jwt.io/introduction>
21. UnityAid. Техническое задание на разработку информационной системы управления волонтерами с элементами геймификации и аналитики. 2026.
22. UnityAid. Описание реализованного функционала по модулям. 2026.
23. UnityAid. Обзор API и ER-диаграмма. 2026.
24. UnityAid. Программа и чеклист испытаний. 2026.
25. UnityAid. Продуктовая стратегия развития системы. 2026.

## ПРИЛОЖЕНИЕ А ФРАГМЕНТ СТРУКТУРЫ БАЗЫ ДАННЫХ

В приложении приведен укрупненный фрагмент логической структуры базы данных UnityAid, отражающий основные связи между сущностями системы.

USERS ||--o{ ORGANIZATION\_MEMBERS : has

ORGANIZATIONS ||--o{ ORGANIZATION\_MEMBERS : includes

ORGANIZATIONS ||--o{ EVENTS : owns

ORGANIZATIONS ||--o{ TASKS : owns

USERS ||--|| VOLUNTEER\_PROFILES : profile

EVENTS ||--o{ EVENT\_APPLICATIONS : receives

EVENTS ||--o{ EVENT\_ATTENDANCE : tracks

TASKS ||--o{ TASK\_ASSIGNMENTS : assigns

USERS ||--o{ TIME\_ENTRIES : submits

USERS ||--o{ VOLUNTEER\_ACHIEVEMENTS : earns

USERS ||--o{ NOTIFICATIONS : receives

USERS ||--o{ CERTIFICATES : receives

ORGANIZATIONS ||--o{ CERTIFICATES : issues

## **ПРИЛОЖЕНИЕ Б ПРОГРАММА ИСПЫТАНИЙ**

Программа испытаний включает проверку backend, frontend и демонстрационного сценария. Backend проверяется запуском тестов, применением миграций и ручной проверкой API. Frontend проверяется сборкой, навигацией, адаптивностью и выполнением основных операций в интерфейсе.

- 1) выполнить проверку миграций базы данных на чистом окружении;
- 2) проверить регистрацию, вход, обновление сессии, выход и смену пароля;
- 3) проверить CRUD организаций, мероприятий, задач, новостей и базы знаний;
- 4) проверить подачу заявки на мероприятие и изменение ее статуса координатором;
- 5) проверить создание, подтверждение и отклонение записей времени;
- 6) проверить начисление достижений и отображение рейтинга;
- 7) проверить генерацию, скачивание и публичную проверку сертификата;
- 8) проверить отображение отчетов аналитики и журнала аудита;
- 9) выполнить сборку frontend и проверить переходы по основному меню.

## ПРИЛОЖЕНИЕ В ФРАГМЕНТ API СИСТЕМЫ

В приложении приведен укрупненный перечень основных endpoint, используемых клиентской частью UnityAid. Фактическая структура API может уточняться при развитии проекта, однако приведенная группировка отражает реализованную архитектуру модулей.

Таблица В.1 - Основные endpoint REST API

| Группа       | Метод и путь                                           | Назначение                                                  |
|--------------|--------------------------------------------------------|-------------------------------------------------------------|
| Авторизация  | POST /api/v1/auth/register                             | Регистрация нового пользователя                             |
| Авторизация  | POST /api/v1/auth/login                                | Вход пользователя в систему                                 |
| Авторизация  | GET /api/v1/auth/me                                    | Получение сведений о текущем пользователе                   |
| Организации  | GET /api/v1/organizations                              | Получение списка доступных организаций                      |
| Организации  | POST /api/v1/organizations                             | Создание организации пользователем с правами администратора |
| Организации  | POST /api/v1/organizations/{id}/members                | Добавление участника в организацию                          |
| Мероприятия  | GET /api/v1/events                                     | Получение списка мероприятий с фильтрами                    |
| Мероприятия  | POST /api/v1/events                                    | Создание мероприятия координатором                          |
| Мероприятия  | POST /api/v1/events/{id}/applications                  | Подача заявки волонтером                                    |
| Мероприятия  | PATCH /api/v1/events/{id}/applications/{applicationId} | Изменение статуса заявки координатором                      |
| Задачи       | GET /api/v1/tasks                                      | Получение списка задач                                      |
| Задачи       | POST /api/v1/tasks                                     | Создание задачи                                             |
| Учет времени | POST /api/v1/time-entries                              | Создание записи волонтерских часов                          |
| Учет времени | PATCH /api/v1/time-entries/{id}/approve                | Подтверждение записи времени                                |
| Аналитика    | GET /api/v1/analytics/overview                         | Получение сводных показателей                               |
| Сертификаты  | POST /api/v1/certificates/generate                     | Генерация сертификата или справки                           |
| Сертификаты  | GET /api/v1/certificates/verify/{code}                 | Публичная проверка сертификата по коду                      |

Пример структуры запроса на создание мероприятия:

```
{
  "organizationId": 1,
  "title": "Помощь в проведении городского мероприятия",
  "format": "offline",
  "startsAt": "2026-06-10T10:00:00+05:00",
```

```
"endsAt": "2026-06-10T15:00:00+05:00",  
"location": "Магнитогорск, площадка мероприятия",  
"capacity": 30  
}
```

Пример ответа API содержит идентификатор созданного мероприятия, текущий статус, сведения об организации и служебные даты создания и изменения. Клиентская часть использует эти данные для перехода на страницу мероприятия и обновления списка.

## **ПРИЛОЖЕНИЕ Г ОПИСАНИЕ АЛГОРИТМОВ ОБРАБОТКИ ЗАЯВОК И ЧАСОВ**

Алгоритм обработки заявки на мероприятие:

- 1) Пользователь открывает опубликованное мероприятие и отправляет заявку.
- 2) Сервер проверяет, что пользователь авторизован и состоит в допустимой организации.
- 3) Сервер проверяет, что мероприятие не отменено и не завершено.
- 4) Сервер проверяет, не существует ли уже активной заявки этого пользователя.
- 5) Если лимит участников не превышен, заявке присваивается статус ожидания рассмотрения или подтверждения согласно настройкам мероприятия.
- 6) Если лимит превышен, заявке может быть присвоен статус листа ожидания.
- 7) Координатор рассматривает заявку и принимает решение.
- 8) Пользователь получает уведомление об изменении статуса.

Алгоритм подтверждения волонтерских часов:

- 1) Волонтер создает запись времени и указывает организацию, дату, длительность и основание.
- 2) Система сохраняет запись со статусом ожидания подтверждения.
- 3) Координатор открывает список записей времени своей организации.
- 4) Координатор проверяет связь записи с мероприятием или задачей.
- 5) При корректных данных координатор подтверждает запись.
- 6) Подтвержденные часы включаются в профиль волонтера и аналитические отчеты.
- 7) При некорректных данных координатор отклоняет запись с комментарием.
- 8) Система фиксирует действие в журнале аудита и отправляет уведомление волонтеру.

Алгоритм начисления достижения:



- 1) После подтверждения часов или завершения мероприятия система получает событие активности.
- 2) Сервис геймификации загружает правила достижений, применимые к организации или системе в целом.
- 3) Для каждого правила рассчитывается, выполнено ли условие пользователем.
- 4) Если достижение еще не выдавалось и условие выполнено, создается запись о выдаче достижения.
- 5) При необходимости создается транзакция баллов.
- 6) Пользователь получает уведомление, а его профиль и рейтинг обновляются.

Алгоритм генерации сертификата:

- 1) Координатор или администратор выбирает пользователя, организацию и период, за который требуется сформировать документ.
- 2) Сервер проверяет права пользователя, запрашивающего генерацию сертификата.
- 3) Система выбирает только подтвержденные записи времени за указанный период.
- 4) Система рассчитывает суммарное количество часов и перечень оснований участия.
- 5) Если подтвержденных данных недостаточно, сервер возвращает сообщение о невозможности сформировать документ.
- 6) При успешной проверке создается запись сертификата с уникальным проверочным кодом.
- 7) Сервер формирует PDF-документ и сохраняет сведения о нем в базе данных.
- 8) Пользователь получает возможность скачать документ, а внешний проверяющий может открыть публичную страницу проверки по коду.

Алгоритм формирования сводной аналитики:

- 1) Пользователь открывает аналитический раздел и выбирает организацию, период и тип отчета.
- 2) Сервер проверяет, что пользователь имеет право просматривать аналитику выбранной организации.
- 3) Система получает агрегированные данные по мероприятиям, задачам, заявкам, подтвержденным часам и достижениям.
- 4) Неподтвержденные записи времени исключаются из итоговых показателей, но могут отображаться отдельно как ожидающие проверки.
- 5) Показатели группируются по периоду, направлению, мероприятию или пользователю в зависимости от выбранного отчета.
- 6) API возвращает структурированный результат, который frontend отображает в виде карточек, таблиц или графиков.
- 7) При необходимости пользователь экспортирует результаты для дальнейшей подготовки управленческого отчета.

Описанные алгоритмы позволяют отделить пользовательское действие от подтвержденного результата. Это особенно важно для учета часов и сертификатов, поскольку документ должен формироваться только на основании проверенных данных.

В проекте такой подход применяется последовательно: заявка еще не означает участие, созданная запись времени еще не означает подтвержденный вклад, а наличие активности еще не означает автоматическую выдачу документа. Между этими состояниями существуют проверки, выполняемые пользователями с соответствующими правами.

Разделение состояний повышает надежность отчетности. Если организация использует UnityAid для официального подтверждения волонтерской активности, то система должна хранить не только итоговое значение часов, но и путь его получения: мероприятие или задачу, пользователя, дату, статус и координатора, подтвердившего запись.

Таким образом, алгоритмы обработки заявок, часов, достижений, сертификатов и аналитики образуют единый контур управления данными. Каждый

следующий этап использует результат предыдущего только после проверки, что снижает риск ошибочных начислений и повышает доверие к итоговым отчетам.

## ПРИЛОЖЕНИЕ Д ЭКРАННЫЕ ФОРМЫ ПРИЛОЖЕНИЯ

В приложении приведены основные экранные формы разработанной информационной системы UnityAid. Скриншоты подтверждают наличие пользовательского интерфейса для авторизации, работы с мероприятиями, задачами, учетом часов, аналитикой, сертификатами и системным администрированием.

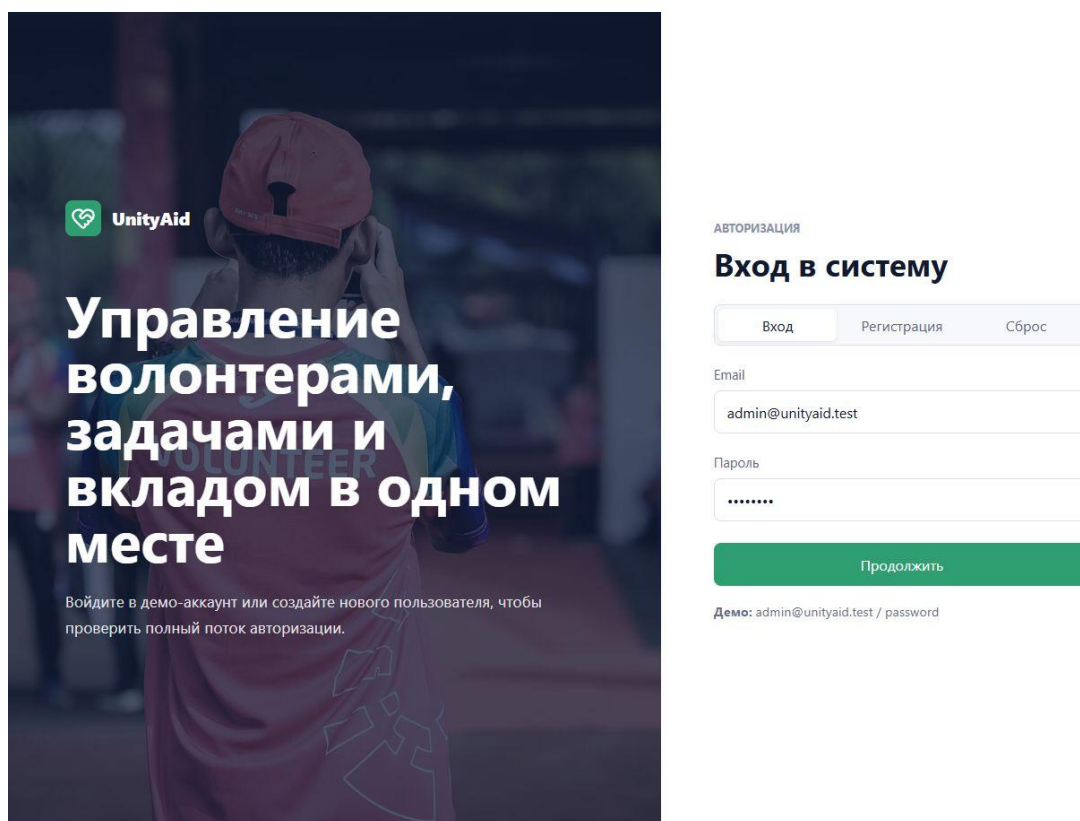


Рисунок Д.1 - Страница авторизации пользователя

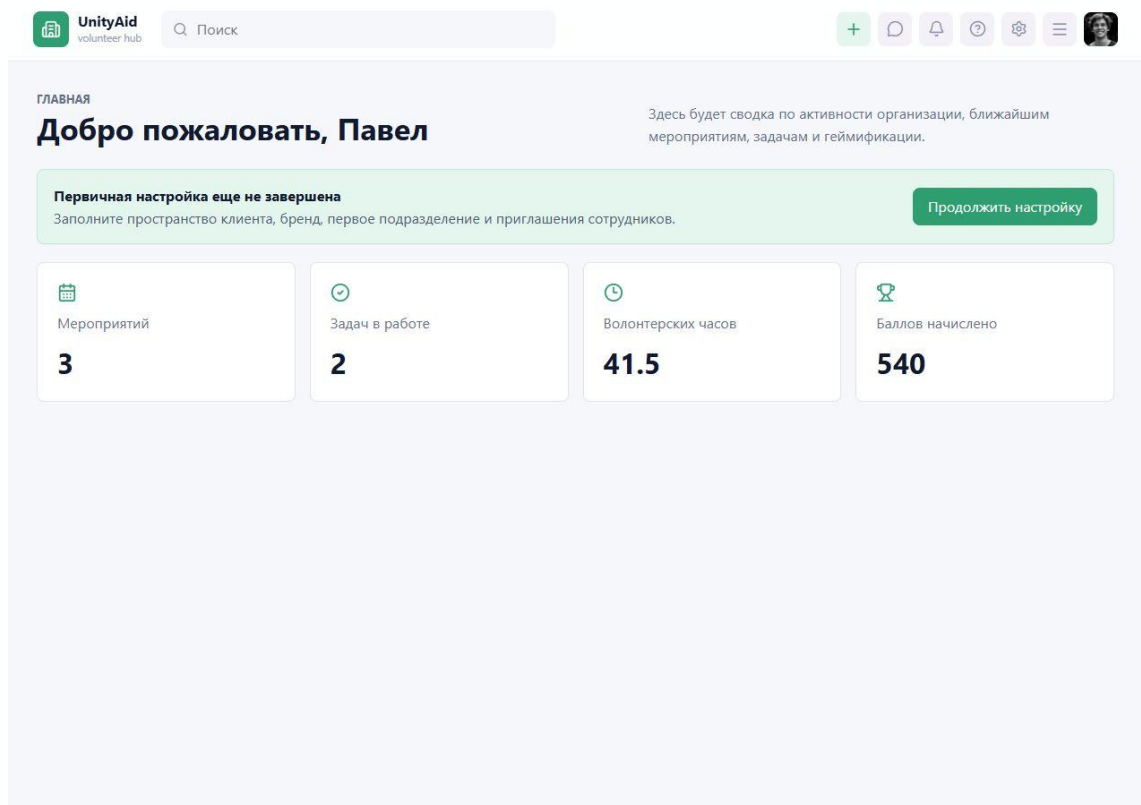


Рисунок Д.2 - Главная панель пользователя

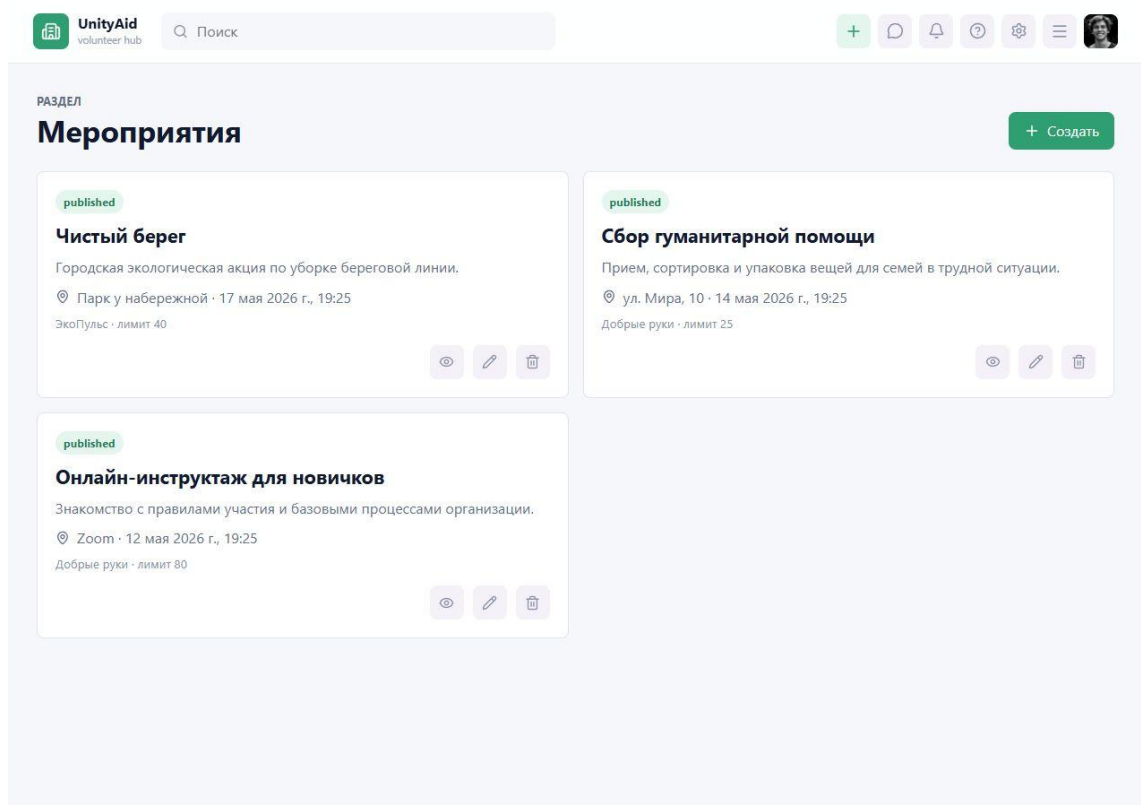


Рисунок Д.3 - Раздел управления мероприятиями

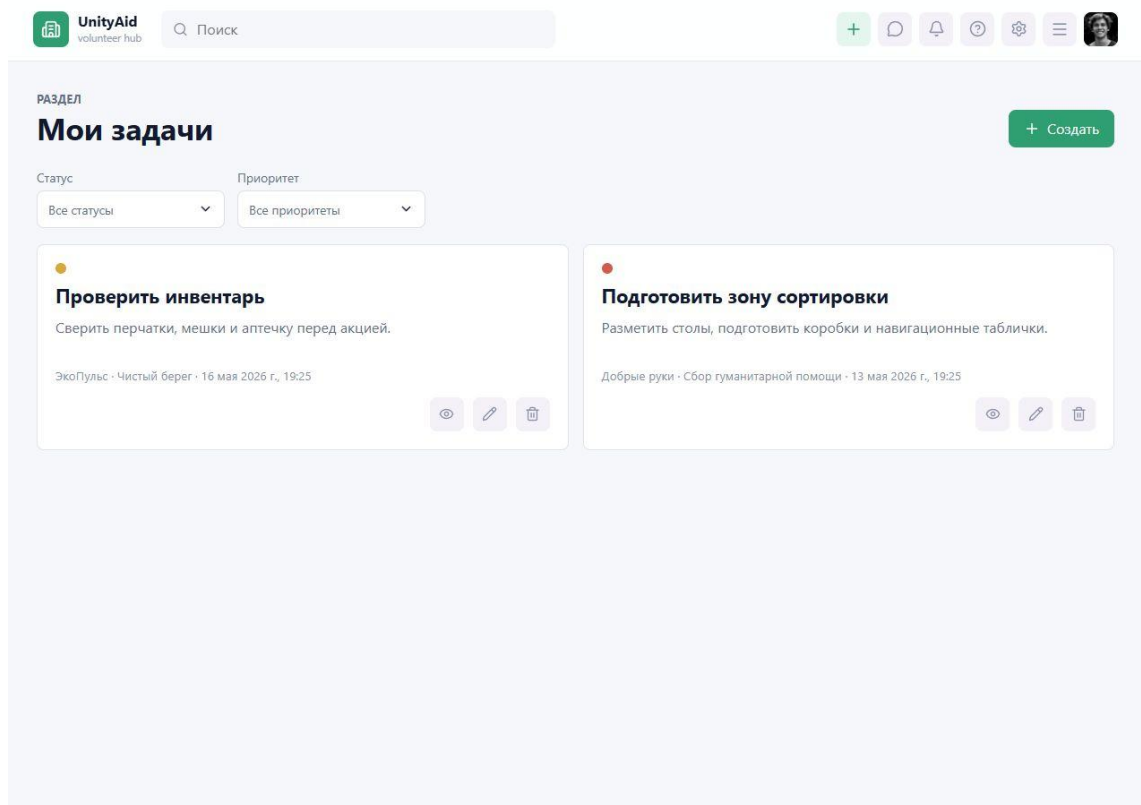


Рисунок Д.4 - Раздел задач

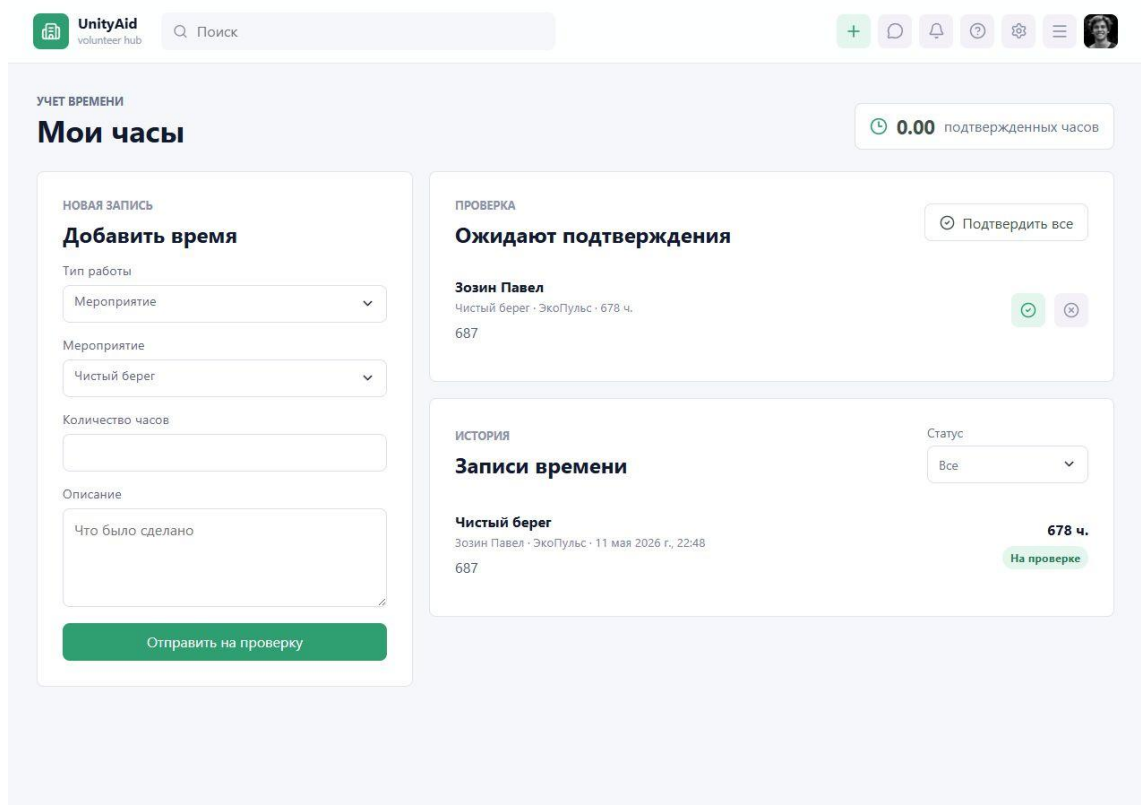


Рисунок Д.5 - Раздел учета волонтерских часов

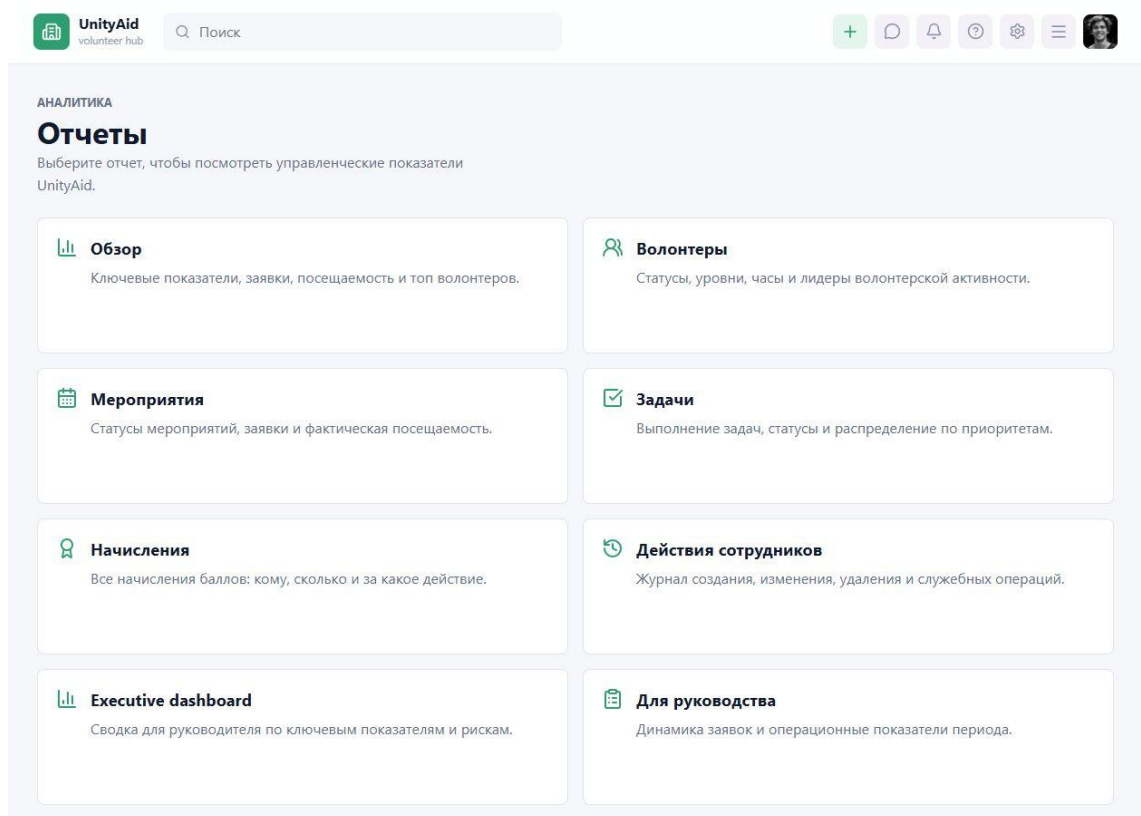


Рисунок Д.6 - Аналитическая панель

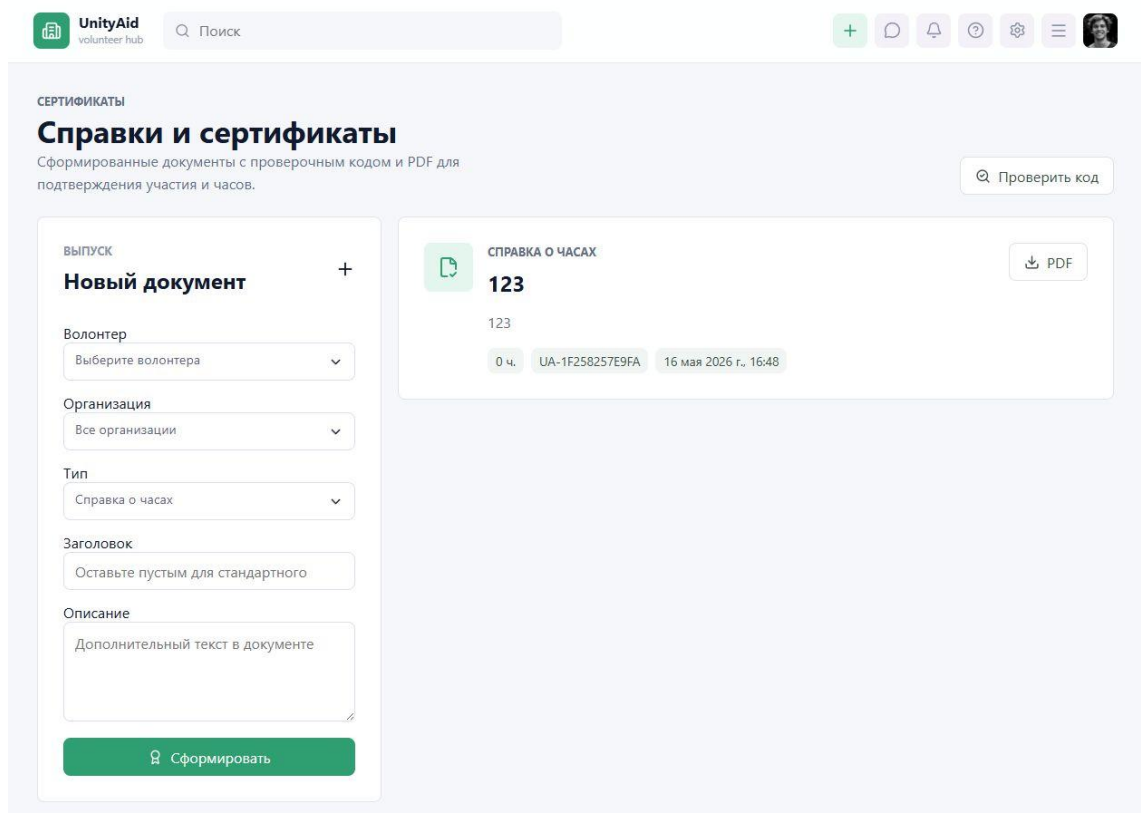


Рисунок Д.7 - Раздел сертификатов

UnityAid

volunteer hub

Поиск

+

АДМИНИСТРИРОВАНИЕ

Системная админ-панель

Постраничное управление сущностями приложения: просмотр, создание, редактирование и удаление.

Обновить

СУЩНОСТИ

Разделы

Аудит

audit\_log

База знаний

knowledge\_articles

Вложения задач

task\_attachments

Достижения

achievements

Достижения волонтеров

volunteer\_achievements

Задачи

tasks

Заявки на мероприятия

event\_applications

AUDIT\_LOG

Аудит

Поиск

| id                                   | user_id                             | method | action | entity_type    | entity_id                | path                          | status_code | ip_address | user_agent                                                                                                                                               | created                    |
|--------------------------------------|-------------------------------------|--------|--------|----------------|--------------------------|-------------------------------|-------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| 87d76f2b-c576-4362-bf3f-b8e1996bd262 | aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaa | POST   | create | certificates   | —                        | /api/v1/certificates/generate | 201         | 172.18.0.1 | Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Codex/26.5 06.31421 Chrome/146.0.7680.179 Electron/41.2.0 Safari/537.36 | 2026-05-16T11:49.644290:00 |
| caaf38c4-8919-4a43-94f5-             | aaaaaaaa-aaaa-aaaa-aaaa-            | PUT    | update | knowledge-base | 479ef1a4-7a98-413a-88b8- | /api/v1/knowledge-base/id     | 200         | 172.18.0.1 | Mozilla/5.0 (Windows NT 10.0;                                                                                                                            | 2026-05-11T18:09.93267     |

Рисунок Д.8 - Системная админ-панель



## ПРИЛОЖЕНИЕ Е ФРАГМЕНТЫ ИСХОДНОГО КОДА

В приложении приведены фрагменты исходного кода, отражающие ключевые части реализации UnityAid: применение миграций базы данных, работу административного API, маршрутизацию клиентского приложения, взаимодействие frontend с REST API и реализацию бизнес-операций.

### Листинг Е.1 - Миграция для пользовательской нумерации записей

```
CREATE OR REPLACE FUNCTION ensure_display_id(target_table regclass)
RETURNS void AS $$
DECLARE
    sequence_name text := replace(target_table::text, '.', '_') || '_display_id_seq';
    index_name text := replace(target_table::text, '.', '_') || '_display_id_key';
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM pg_attribute
        WHERE attrelid = target_table
            AND attname = 'display_id'
            AND NOT attisdropped
    ) THEN
        EXECUTE format('ALTER TABLE %s ADD COLUMN display_id BIGINT', target_table);
    END IF;

    EXECUTE format('CREATE SEQUENCE IF NOT EXISTS %I', sequence_name);
    EXECUTE format('ALTER SEQUENCE %I OWNED BY %s.display_id', sequence_name, target_table);
    EXECUTE format('ALTER TABLE %s ALTER COLUMN display_id SET DEFAULT nextval(%L::regclass)',
target_table, sequence_name);

    EXECUTE format(
        'WITH numbered AS (
            SELECT ctid, row_number() OVER (ORDER BY ctid) AS rn
            FROM %s
            WHERE display_id IS NULL
        )
        UPDATE %s AS t
        SET display_id = numbered.rn
        FROM numbered
        WHERE t.ctid = numbered.ctid',
target_table,
target_table
    );

    EXECUTE format(
        'SELECT setval(%L, GREATEST(COALESCE((SELECT max(display_id) FROM %s), 0) + 1, 1),
false)',
sequence_name,
target_table
    );

    EXECUTE format('ALTER TABLE %s ALTER COLUMN display_id SET NOT NULL', target_table);
    EXECUTE format('CREATE UNIQUE INDEX IF NOT EXISTS %I ON %s (display_id)', index_name,
target_table);
END;
$$ LANGUAGE plpgsql;

SELECT ensure_display_id('organizations'::regclass);
SELECT ensure_display_id('users'::regclass);
SELECT ensure_display_id('organization_members'::regclass);
SELECT ensure_display_id('volunteer_profiles'::regclass);
SELECT ensure_display_id('skills'::regclass);
SELECT ensure_display_id('volunteer_skills'::regclass);
SELECT ensure_display_id('events'::regclass);
SELECT ensure_display_id('tasks'::regclass);
SELECT ensure_display_id('task_assignments'::regclass);
SELECT ensure_display_id('news'::regclass);
SELECT ensure_display_id('refresh_sessions'::regclass);
SELECT ensure_display_id('revoked_access_tokens'::regclass);
```

```

SELECT ensure_display_id('email_verification_tokens'::regclass);
SELECT ensure_display_id('password_reset_tokens'::regclass);
SELECT ensure_display_id('event_applications'::regclass);
SELECT ensure_display_id('event_attendance'::regclass);
SELECT ensure_display_id('event_shifts'::regclass);
SELECT ensure_display_id('event_feedback'::regclass);
SELECT ensure_display_id('task_comments'::regclass);
SELECT ensure_display_id('task_attachments'::regclass);
SELECT ensure_display_id('task_status_history'::regclass);
SELECT ensure_display_id('task_time_entries'::regclass);
SELECT ensure_display_id('news_categories'::regclass);
SELECT ensure_display_id('system_roles'::regclass);
SELECT ensure_display_id('user_system_roles'::regclass);

```

## Листинг Е.2 - Формирование списка записей в административном модуле

### backend

```

func (r *Repository) List(ctx context.Context, code string, page int, limit int, search string)
(ListResponse, error) {
    config, ok := entityConfigs[code]
    if !ok {
        return ListResponse{}, ErrUnknownEntity
    }
    config = withDisplayID(config)
    if page < 1 {
        page = 1
    }
    if limit < 1 || limit > 100 {
        limit = 25
    }
    where, args := searchWhere(config, search)
    var total int
    if err := r.db.QueryRow(ctx, "SELECT COUNT(*) FROM "+config.Table+where,
args...).Scan(&total); err != nil {
        return ListResponse{}, err
    }
    args = append(args, limit, (page-1)*limit)
    rows, err := r.db.Query(ctx, "SELECT row_to_json(t)::text FROM (SELECT
"+strings.Join(config.Columns, ", ")+" FROM "+config.Table+where
    if err != nil {
        return ListResponse{}, err
    }
    defer rows.Close()
    items := []map[string]any{}
    for rows.Next() {
        var raw string
        if err := rows.Scan(&raw); err != nil {
            return ListResponse{}, err
        }
        item := map[string]any{}
        if err := json.Unmarshal([]byte(raw), &item); err != nil {
            return ListResponse{}, err
        }
        items = append(items, item)
    }
    return ListResponse{Items: items, Total: total, Page: page, Limit: limit}, rows.Err()
}

```

## Листинг Е.3 - Клиентские функции административного REST API

```

import { apiRequest } from '../shared/api'
import { authState } from '../auth/store'
import type { AdminEntitiesResponse, AdminEntityRow, AdminListResponse } from './types'

const token = () => authState.token

export function fetchAdminEntities() {
    return apiRequest<AdminEntitiesResponse>('/admin/entities', { token: token() })
}

```

```

export function fetchAdminRows(entity: string, params: { page?: number; perPage?: number; search?: string } = {}) {
  const query = new URLSearchParams()
  if (params.page) query.set('page', String(params.page))
  if (params.perPage) query.set('perPage', String(params.perPage))
  if (params.search) query.set('search', params.search)
  const suffix = query.toString() ? `?${query.toString()}` : ''
  return apiRequest<AdminListResponse>(`/admin/${entity}${suffix}`, { token: token() })
}

export function createAdminRow(entity: string, data: AdminEntityRow) {
  return apiRequest<{ item: AdminEntityRow }>(`/admin/${entity}`, {
    method: 'POST',
    token: token(),
    body: JSON.stringify({ data })
  })
}

export function updateAdminRow(entity: string, id: string, data: AdminEntityRow) {
  return apiRequest<{ item: AdminEntityRow }>(`/admin/${entity}/${id}`, {
    method: 'PUT',
    token: token(),
    body: JSON.stringify({ data })
  })
}

export function deleteAdminRow(entity: string, id: string) {
  return apiRequest<void>(`/admin/${entity}/${id}`, {
    method: 'DELETE',
    token: token()
  })
}

```

## Листинг Е.4 - Регистрация маршрутов клиентского приложения

```

import { createRouter, createWebHistory } from 'vue-router'
import { authState, fetchCurrentUser } from '../entities/auth/store'
import AppLayout from '../widgets/layout/AppLayout.vue'
import DashboardPage from '../pages/DashboardPage.vue'
import LoginPage from '../pages/LoginPage.vue'
import NewsDetailPage from '../pages/NewsDetailPage.vue'
import NewsFormPage from '../pages/NewsFormPage.vue'
import NewsListPage from '../pages/NewsListPage.vue'
import NotificationsPage from '../pages/NotificationsPage.vue'
import OnboardingPage from '../pages/OnboardingPage.vue'
import OrganizationsPage from '../pages/OrganizationsPage.vue'
import ProfilePage from '../pages/ProfilePage.vue'
import EventsPage from '../pages/EventsPage.vue'
import KnowledgeArticlePage from '../pages/KnowledgeArticlePage.vue'
import KnowledgeBasePage from '../pages/KnowledgeBasePage.vue'
import KnowledgeFormPage from '../pages/KnowledgeFormPage.vue'
import TasksPage from '../pages/TasksPage.vue'
import EventDetailPage from '../pages/EventDetailPage.vue'
import AchievementsPage from '../pages/AchievementsPage.vue'
import AnalyticsPage from '../pages/AnalyticsPage.vue'
import AdminPanelPage from '../pages/AdminPanelPage.vue'
import CertificatesPage from '../pages/CertificatesPage.vue'
import CertificateVerifyPage from '../pages/CertificateVerifyPage.vue'
import OrganizationDetailPage from '../pages/OrganizationDetailPage.vue'
import TaskDetailPage from '../pages/TaskDetailPage.vue'
import TimeEntriesPage from '../pages/TimeEntriesPage.vue'
import VolunteersPage from '../pages/VolunteersPage.vue'
import SettingsPage from '../pages/SettingsPage.vue'

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    {
      path: '/login',
      name: 'login',
      component: LoginPage,
      meta: { public: true }
    },
  ],

```

```

{
  path: '/',
  component: AppLayout,
  children: [
    {
      path: '',
      name: 'dashboard',
      component: DashboardPage
    },
    {
      path: 'profile',
      name: 'profile',
      component: ProfilePage
    },
    {
      path: 'onboarding',
      name: 'onboarding',
      component: OnboardingPage
    },
    {
      path: 'settings',
      name: 'settings',
      component: SettingsPage
    },
    {
      path: 'settings/:section',
      name: 'settings-section',
      component: SettingsPage
    },
    {
      path: 'admin',
      name: 'admin',
      component: AdminPanelPage
    },
    {
      path: 'organizations',
      name: 'organizations',
      component: OrganizationsPage
    },
    {
      path: 'organizations/:id',
      name: 'organization-detail',
      component: OrganizationDetailPage
    },
    {
      path: 'volunteers',
      name: 'volunteers',

```

## Листинг Е.5 - Обработка заявок на мероприятия

```

func (r *Repository) CreateApplication(ctx context.Context, eventID string, userID string, message
string) (Application, error) {
  var existingID string
  err := r.db.QueryRow(ctx, `SELECT id::text FROM event_applications WHERE event_id = $1 AND
user_id = $2`, eventID, userID).Scan(&exist
if err == nil {
  return Application{}, ErrAlreadyExists
}
if !errors.Is(err, pgx.ErrNoRows) {
  return Application{}, err
}
status, err := r.nextApplicationStatus(ctx, eventID)
if err != nil {
  return Application{}, err
}
var id string
err = r.db.QueryRow(ctx, `
  INSERT INTO event_applications (event_id, user_id, status, message)
  VALUES ($1, $2, $3, $4)
  RETURNING id::text
`, eventID, userID, status, message).Scan(&id)
if err != nil {
  return Application{}, err
}
return r.findApplication(ctx, eventID, id)
}

```

## Листинг Е.6 - Сбор аналитических показателей

```
func (r *Repository) Overview(ctx context.Context, filters Filters) (OverviewReport, error) {
    metrics, err := r.metrics(ctx, filters)
    if err != nil {
        return OverviewReport{}, err
    }
    applications, err := r.series(ctx, `event_applications`, `created_at`, filters)
    if err != nil {
        return OverviewReport{}, err
    }
    attendance, err := r.series(ctx, `event_attendance`, `created_at`, filters)
    if err != nil {
        return OverviewReport{}, err
    }
    top, err := r.topVolunteers(ctx)
    if err != nil {
        return OverviewReport{}, err
    }
    return OverviewReport{Metrics: metrics, Applications: applications, Attendance: attendance,
        TopVolunteers: top}, nil
}

func (r *Repository) Volunteers(ctx context.Context, filters Filters) (VolunteersReport, error) {
    metrics, err := r.volunteerMetrics(ctx, filters)
    if err != nil {
        return VolunteersReport{}, err
    }
    status, err := r.group(ctx, `SELECT vp.status::text, COUNT(*)::float8 FROM
volunteer_profiles vp GROUP BY vp.status::text ORDER BY 1`)
    if err != nil {
        return VolunteersReport{}, err
    }
    hoursByLevel, err := r.group(ctx, `SELECT CONCAT('Уровень ', level),
SUM(total_hours)::float8 FROM volunteer_profiles GROUP BY level 0`)
    if err != nil {
        return VolunteersReport{}, err
    }
    top, err := r.topVolunteers(ctx)
    if err != nil {
        return VolunteersReport{}, err
    }
    return VolunteersReport{Metrics: metrics, Status: status, HoursByLevel: hoursByLevel,
        TopVolunteers: top}, nil
}

func (r *Repository) Events(ctx context.Context, filters Filters) (EventsReport, error) {
    metrics, err := r.eventMetrics(ctx, filters)
    if err != nil {
        return EventsReport{}, err
    }
    byStatus, err := r.group(ctx, `SELECT status::text, COUNT(*)::float8 FROM events GROUP BY
status::text ORDER BY 1`)
    if err != nil {
        return EventsReport{}, err
    }
    applications, err := r.series(ctx, `event_applications`, `created_at`, filters)
    if err != nil {
        return EventsReport{}, err
    }
    attendance, err := r.series(ctx, `event_attendance`, `created_at`, filters)
    if err != nil {
        return EventsReport{}, err
    }
    return EventsReport{Metrics: metrics, ByStatus: byStatus, Applications: applications,
        Attendance: attendance}, nil
}

func (r *Repository) Tasks(ctx context.Context, filters Filters) (TasksReport, error) {
    metrics, err := r.taskMetrics(ctx, filters)
    if err != nil {
        return TasksReport{}, err
    }
}
```

```

        byStatus, err := r.group(ctx, `SELECT status::text, COUNT(*)::float8 FROM tasks GROUP BY
status::text ORDER BY 1`)
        if err != nil {
            return TasksReport{}, err
        }
        byPriority, err := r.group(ctx, `SELECT priority, COUNT(*)::float8 FROM tasks GROUP BY
priority ORDER BY priority`)
        if err != nil {
            return TasksReport{}, err
        }
        completed, err := r.series(ctx, `tasks`, `completion_confirmed_at`, filters)
        if err != nil {

```